BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC KINH TẾ TP HỒ CHÍ MINH (UEH)

TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

ĐỒ ÁN MÔN HỌC

ĐỀ TÀI

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG CHIA SẺ KHÔNG GIAN BROOMSHARING

Học Phần: Phát Triển Ứng Dụng Mobile

Danh Sách Nhóm:

1 LỮ VÕ HOÀNG PHÚC – 31241025389

2 LÂM HOÀNG – *[Chưa cung cấp MSSV]*

3 TỪ TRƯƠNG THANH TRÚC – *[Chưa cung cấp MSSV]*

Chuyên Ngành: KỸ THUẬT PHẦN MỀM

Khóa: K50

Giảng Viên: Đặng Ngọc Hoàng Thành

Tp. Hồ Chí Minh, tháng 10 năm 2026

ABSTRACT / TÓM TẮT

BroomSharing là đề tài ứng dụng Android hướng đến việc kết nối những người có không gian chưa khai thác hết với người cần thuê hoặc sử dụng không gian linh hoạt. Bài toán tập trung vào tính phân tán của tin đăng, khó khăn khi so sánh vị trí, giá, tiện ích, thời gian sẵn có và việc theo dõi yêu cầu đặt thuê. Hệ thống gồm ba vai trò người thuê, Host và Admin, với các luồng khám phá, đăng tin, xét duyệt, đặt chỗ và quản lý lượt sử dụng.

Báo cáo trình bày phân tích nghiệp vụ, thiết kế giao diện và cấu trúc lưu trữ Cloud Firestore của ứng dụng Java + XML theo MVVM kết hợp Repository và Model. Hệ thống kết nối Firebase Authentication, Cloudinary, bản đồ và kênh thanh toán payOS demo. Các sơ đồ mô tả quy trình giữ lịch, chuyển trạng thái booking và phân quyền; bộ prototype giao diện được phân biệt với phần hiện thực và kết quả kiểm thử.

Từ khóa: BroomSharing; Android; Java; XML; MVVM; Repository; Cloud Firestore; chia sẻ không gian; đặt thuê.

DANH MỤC VIẾT TẮT VÀ THUẬT NGỮ

Chữ viết tắt

Ký hiệu	Tiếng Anh	Giải thích
MVVM	Model–View–ViewModel	Kiến trúc tách giao diện, trạng thái UI và mô hình dữ liệu.
UI	User Interface	Giao diện người dùng.
SDK	Software Development Kit	Bộ công cụ phát triển.
API	Application Programming Interface	Giao diện lập trình ứng dụng.
XML	Extensible Markup Language	Ngôn ngữ khai báo bố cục Android.
POJO	Plain Old Java Object	Đối tượng Java biểu diễn dữ liệu.
DTO	Data Transfer Object	Đối tượng truyền dữ liệu.
CRUD	Create, Read, Update, Delete	Tạo, đọc, sửa, xóa dữ liệu.
QR	Quick Response	Mã phản hồi nhanh.

Thuật ngữ chuyên ngành

Thuật ngữ	Diễn giải trong đề tài
Activity / Fragment	Thành phần View của Android, nhận tương tác và hiển thị trạng thái.
ViewModel / LiveData	Quản lý dữ liệu cho màn hình và cho phép View quan sát thay đổi theo vòng đời.
Repository	Thành phần tập trung các thao tác truy cập và ghi dữ liệu.
Model / Entity	Lớp Java mô tả dữ liệu như `UserProfile`, `Room`, `Booking`.
Cloud Firestore	Dịch vụ lưu trữ NoSQL theo collection / document.
Booking	Yêu cầu sử dụng một không gian trong khoảng thời gian xác định.
Host	Tài khoản được cấp quyền cho thuê, quản lý các tòa nhà và phòng thuộc sở hữu.
Check-in	Bước xác nhận nhận hoặc bắt đầu sử dụng không gian.
Geocoding	Chuyển đổi địa chỉ hoặc tên địa điểm thành tọa độ.
Transaction	Thao tác xử lý dữ liệu có kiểm soát tính nhất quán.

MỤC LỤC

ABSTRACT / TÓM TẮT

DANH MỤC VIẾT TẮT VÀ THUẬT NGỮ

1 TỔNG QUAN **1**

1.1 Giới thiệu 1

1.2 Tính cấp thiết của đề tài 1

1.3 Mục tiêu và phạm vi đề tài 1

1.4 Công cụ phát triển ứng dụng di động 2

1.5 Kiểm thử trên Android Emulator và thiết bị 3

2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **4**

2.1 Tác nhân và yêu cầu chức năng 4

2.2 Sơ đồ công việc 4

2.3 Sơ đồ phân rã chức năng 10

2.4 Sơ đồ use case 10

2.5 Đặc tả các nghiệp vụ tiêu biểu 15

2.6 Yêu cầu phi chức năng và các ràng buộc thiết kế 16

3 THIẾT KẾ GIAO DIỆN **17**

3.1 Tổng quan về thiết kế giao diện 17

3.2 Giao diện phân hệ người thuê 18

3.3 Giao diện phân hệ chủ không gian 21

3.4 Giao diện phân hệ quản trị viên 22

4 THIẾT KẾ CƠ SỞ DỮ LIỆU **23**

4.1 Các thực thể và thuộc tính 23

4.2 Quan hệ dữ liệu và cấu trúc lưu trữ 24

4.3 Một số thao tác dữ liệu tiêu biểu 31

4.4 Phân quyền và bảo vệ dữ liệu 34

5 XÂY DỰNG ỨNG DỤNG VÀ THIẾT KẾ KIẾN TRÚC **36**

5.1 Tổng quan kiến trúc Java + XML, MVVM + Repository + Model 36

5.2 Cấu trúc thư mục đề xuất 37

5.3 Kết nối với Firebase 37

5.4 Minh họa luồng nghiệp vụ “xem danh sách không gian” 37

5.5 Tích hợp bản đồ và ảnh trong kiến trúc 38
5.6 Các màn hình và mức độ hoàn thiện 38

6 QUY TRÌNH THỰC THI VÀ KIỂM THỬ 39
6.1 Quy trình đăng ký và đăng nhập 39
6.2 Quy trình tìm kiếm và đặt thuê không gian 39
6.3 Quy trình đăng và quản lý không gian 39
6.4 Quy trình xác nhận và QR check-in 39
6.5 Thiết kế các ca kiểm thử 40
6.6 Đánh giá giới hạn và chứng cứ thực thi 41

7 KẾT LUẬN VÀ HƯỚNG MỞ RỘNG 42
7.1 Kết luận của giai đoạn thiết kế 42
7.2 Những hạn chế hiện tại 42
7.3 Hướng phát triển 42

TÀI LIỆU THAM KHẢO 43

PHỤ LỤC A. HÌNH ẢNH 45

PHỤ LỤC B. TRUY VẤN SQL VÀ FIRESTORE 48

PHỤ LỤC C. DANH MỤC BẢNG, BIỂU 50

PHỤ LỤC D. ĐƯỜNG DẪN TÀI NGUYÊN ĐỒ ÁN 51

PHỤ LỤC E. BẢNG PHÂN CÔNG VÀ MINH CHỨNG NHÓM 52

PHỤ LỤC F. TƯ LIỆU VÀ KIỂM TRA BỔ SUNG 53

PHỤ LỤC G. PHÂN RÃ CHỨC NĂNG CHI TIẾT 54

1. TỔNG QUAN

1.1 Giới thiệu

Trong lĩnh vực dịch vụ thương mại điện tử và kinh tế chia sẻ, mô hình kết nối nhu cầu sử dụng tài nguyên giúp khai thác không gian chưa được sử dụng hết công suất. Đối với chỗ ngồi cá nhân, phòng họp và văn phòng, người thuê cần so sánh vị trí, giá, tiện ích và lịch trống trước khi đặt chỗ.

Bài toán nghiên cứu của đề tài là xây dựng ứng dụng di động BroomSharing, hỗ trợ kết nối trực tiếp giữa khách thuê (Guest) có nhu cầu sử dụng không gian, chủ không gian (Host) muốn khai thác mặt bằng và quản trị viên (Admin) thực hiện kiểm duyệt, xử lý các vấn đề phát sinh. Ứng dụng tổ chức hoạt động tìm kiếm, đặt chỗ, trao đổi và theo dõi lượt thuê trên cùng một hệ thống.

1.2 Tính cấp thiết của đề tài

Việc tìm kiếm và đặt chỗ qua các kênh trao đổi rời rạc đặt ra một số khó khăn: thông tin giá và tiện ích thiếu thống nhất; người thuê phải liên hệ để xác nhận lịch trống; chủ không gian khó theo dõi đồng thời nhiều yêu cầu. Khi dữ liệu lịch không được cập nhật nhất quán, các yêu cầu có thể trùng thời gian trên cùng một phòng.

Vì vậy, BroomSharing tập trung chuẩn hóa bài đăng, kiểm tra khả dụng và quản lý vòng đời booking. Phòng đang hiển thị chưa chắc còn trống ở mọi thời điểm; booking chỉ được xác nhận sau khi hệ thống ghi nhận đủ khoản trả trước trong thời hạn thanh toán. Hệ thống phải xác định ai giữ lịch, lượt nào đã sử dụng và trường hợp nào cần Admin can thiệp, đồng thời theo dõi riêng trạng thái tiền và trạng thái sử dụng.

1.3 Mục tiêu và phạm vi đề tài

Mục tiêu của đề tài là xây dựng ứng dụng Android bằng Java và XML theo MVVM kết hợp Repository và Model. Ứng dụng cung cấp thông tin tìm thuê, thống nhất quy trình quản lý phòng và lưu căn cứ xử lý cho Admin. Dữ liệu cần nhất quán khi nhiều tài khoản thao tác, với kết quả xử lý được thể hiện rõ trên màn hình.

Mỗi phòng cho thuê theo giờ hoặc theo kỳ một tháng, ba tháng. Lịch được tạm giữ tối đa 5 phút khi khởi tạo thanh toán; booking được lưu lịch sử sau khi khép. Host chỉ đổi chế độ thuê và cấu hình lịch khi phòng không còn booking giữ chỗ.

Thanh toán sử dụng một kênh payOS demo cho khoản trả trước 40%; QR được mở ngay khi đặt chỗ, không qua bước Host duyệt booking, chưa có hoàn tiền tự động, phân chia tiền cho

nhiều Host hoặc hợp đồng điện tử. Ảnh giấy tờ xét duyệt Host là dữ liệu DEMO, không tương đương định danh pháp lý.

1.4 Công cụ phát triển ứng dụng di động

1.4.1 Java và XML

Java xử lý giao diện, trạng thái và nghiệp vụ; XML định nghĩa bố cục và tài nguyên hiển thị. Việc phân tách View, ViewModel và Repository giới hạn trách nhiệm từng thành phần, phù hợp với định hướng phân lớp giao diện và dữ liệu trong kiến trúc Android (Android Developers, n.d.-c).

1.4.2 Android SDK và Android Studio

Android SDK cung cấp API và công cụ xây dựng ứng dụng. Android Studio hỗ trợ biên tập Java/XML, quản lý tài nguyên, biên dịch bằng Gradle và theo dõi lỗi qua Logcat; Android Emulator chạy ứng dụng trên thiết bị ảo.

1.4.3 Firebase và Cloud Firestore

Firebase cung cấp các dịch vụ xác thực và dữ liệu cho ứng dụng. Firebase Authentication quản lý danh tính đăng nhập, còn Cloud Firestore lưu hồ sơ người dùng, phòng, booking và dữ liệu tương tác. Cloud Firestore là cơ sở dữ liệu NoSQL hướng tài liệu: dữ liệu được tổ chức thành collection chứa document, thay cho các bảng và hàng của mô hình quan hệ (Firebase, n.d.-b).

Document gồm các trường dữ liệu, có thể chứa map, array hoặc gắn với subcollection. Trong BroomSharing, các đối tượng còn liên kết bằng ID như `roomId` và `ownerUid`. Vì vậy, mô hình ở Chương 4 thể hiện cây collection, document, dữ liệu nhúng và quan hệ logic giữa các đối tượng. Cấu trúc linh hoạt vẫn được kiểm soát bằng quy ước dữ liệu trong Model, Repository và Security Rules; việc tạo booking phải nhất quán với lịch giữ chỗ.

1.4.4 Lưu trữ ảnh, bản đồ và thanh toán

Cloudinary lưu ảnh phòng, ảnh bìa, ảnh đại diện và tệp minh chứng DEMO; Firestore giữ URL tham chiếu (Cloudinary, n.d.). Bản đồ sử dụng MapLibre, kết hợp MapTiler và openrouteservice để tìm địa điểm và tuyến đường.

Backend Java xác thực người dùng, tạo yêu cầu payOS và nhận kết quả từ webhook chuyển tiếp. Ứng dụng quan sát trạng thái tiền để cập nhật màn hình, không tự chuyển sang nhận phòng hoặc hoàn tất lượt thuê.

1.5 Kiểm thử trên Android Emulator và thiết bị

Android Emulator hỗ trợ kiểm tra biểu mẫu, điều hướng và trạng thái; thiết bị thực bổ sung tải ảnh, vị trí và kết nối mạng. Kiểm thử đối chiếu màn hình với dữ liệu, tập trung vào đặt trùng lịch, hủy booking, phân quyền và thanh toán, kèm kịch bản cùng bằng chứng.

2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Tác nhân và yêu cầu chức năng

BroomSharing tổ chức nghiệp vụ theo ba tác nhân: người thuê, Host và Admin. Người thuê tìm kiếm và gửi yêu cầu sử dụng không gian; Host quản lý tài sản và xử lý lượt thuê; Admin kiểm duyệt và giải quyết các trường hợp vượt quyền của hai bên. Các yêu cầu chức năng được nhóm theo trách nhiệm tại Bảng 2.1.1.

Bảng 2.1.1: Tác nhân và phạm vi chức năng

Tác nhân	Chức năng chính
Người thuê	Quản lý tài khoản; tìm, lọc và xem phòng trên bản đồ; lưu yêu thích; đặt, hủy và theo dõi booking; thanh toán; chat; đánh giá và báo cáo vi phạm; gửi đơn làm Host.
Host	Quản lý tòa nhà và phòng; thiết lập giá, lịch; gửi xét duyệt và mở cho thuê; xử lý booking, nhận phòng, hoàn tất hoặc ghi nhận không đến; gửi minh chứng và phản hồi báo cáo. Host vẫn có thể thuê phòng của người khác.
Admin	Duyệt đơn Host, minh chứng và bài đăng; đình chỉ phòng, khóa Host; xử lý báo cáo, booking của Host bị khóa; xác minh việc sử dụng, đối soát lịch giữ chỗ và tra cứu nhật ký.

Host là vai trò có thêm quyền cho thuê sau khi hồ sơ được duyệt. Quyền Host không đồng nghĩa bài đăng được công bố ngay: từng phòng còn phải qua xét duyệt và được chủ phòng mở cho thuê. Dịch vụ thanh toán là tác nhân bên ngoài, tiếp nhận giao dịch và trả kết quả cho hệ thống.

2.2 Sơ đồ công việc

2.2.1 Đăng ký Host và công bố không gian

Người dùng gửi đơn làm Host kèm thông tin và hai ảnh giấy tờ DEMO. Admin xem hồ sơ trước khi chấp thuận hoặc từ chối. Khi được cấp quyền, Host tạo tòa nhà, bổ sung phòng, cấu hình giá, lịch và gửi bài đăng xét duyệt. Phòng chỉ xuất hiện trong danh sách cho thuê khi vừa được duyệt vừa ở trạng thái hoạt động. Luồng tại Hình 2.2.1 tách riêng quyết định cấp quyền Host và quyết định duyệt phòng.

2.2.2 Tạo và xử lý yêu cầu đặt chỗ

Người thuê chọn phòng, thời gian và kiểm tra giá trước khi gửi yêu cầu. Hệ thống kiểm tra trạng thái phòng, quyền của các bên và lịch khả dụng; nếu hợp lệ, booking được tạo ở trạng thái `PENDING`. Booking, lịch giữ chỗ, thông tin cập nhật của phòng và nhật ký được ghi trong cùng transaction. Lịch này là phần giữ chỗ tạm trong tối đa 5 phút, không vượt giờ bắt đầu thuê; yêu

cầu khác không được chiếm cùng khoảng thời gian trong lúc giữ chỗ còn hiệu lực.

Hình 2.2.2 thể hiện các nhánh xử lý sau khi tạo yêu cầu. Ứng dụng mở QR ngay sau bước tạo booking. Backend ghi nhận đủ 40% đúng hạn mới chuyển booking sang `CONFIRMED`; Host không duyệt booking mới. Khi hết hạn hoặc hủy, backend đối chiếu trạng thái của payOS trước khi kết thúc link chưa trả, nhả lịch và giữ lịch sử. Nếu chưa xác định được kết quả từ nhà cung cấp, lịch chưa được nhả cho đến khi đối soát được kết quả.

2.2.3 Sử dụng phòng và xử lý ngoại lệ

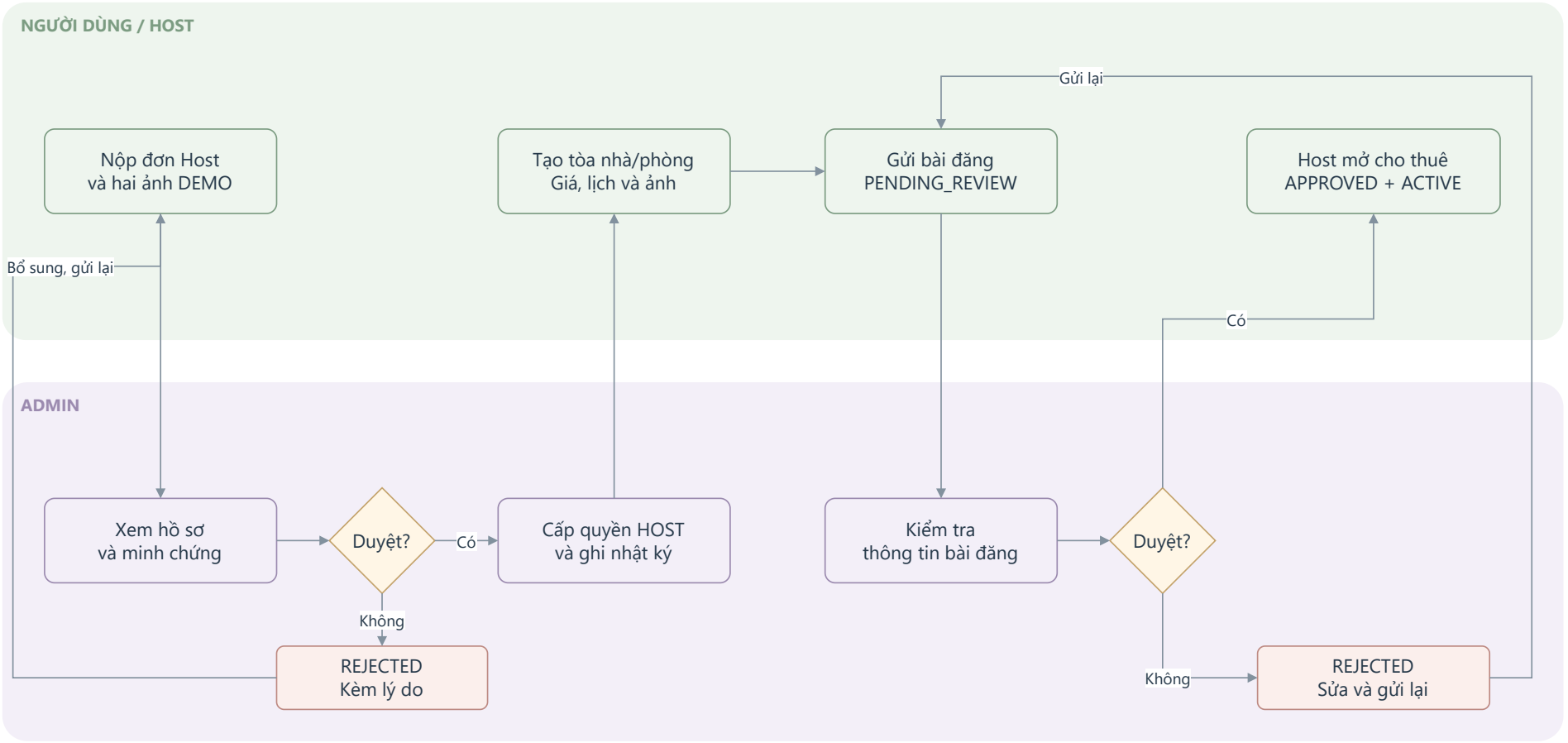
Booking đã xác nhận được Host ghi nhận việc nhận phòng trong khoảng thời gian thuê và hoàn tất sau giờ kết thúc. Nếu khách không đến, Host ghi nhận `NO_SHOW` sau giờ kết thúc, kèm lý do. Trường hợp đã sử dụng nhưng thiếu thao tác nhận phòng cần có xác nhận sử dụng trước khi hoàn tất; thời gian trôi qua không tự chứng minh khách đã dùng phòng.

Với Host bị khóa, Admin xử lý từng booking thay vì hủy hàng loạt. Booking chưa nhận phòng có thể bị hủy với lý do; lượt đã nhận phòng có thể kết thúc do sự cố ở trạng thái `TERMINATED`. Việc sửa kết luận không đến hoặc bổ sung xác nhận sử dụng phải có nhật ký, không nhả lịch lần thứ hai. Chỉ người thuê của booking đã khép và có xác nhận sử dụng mới được đánh giá. Quy trình chi tiết được thể hiện tại Hình 2.2.3.

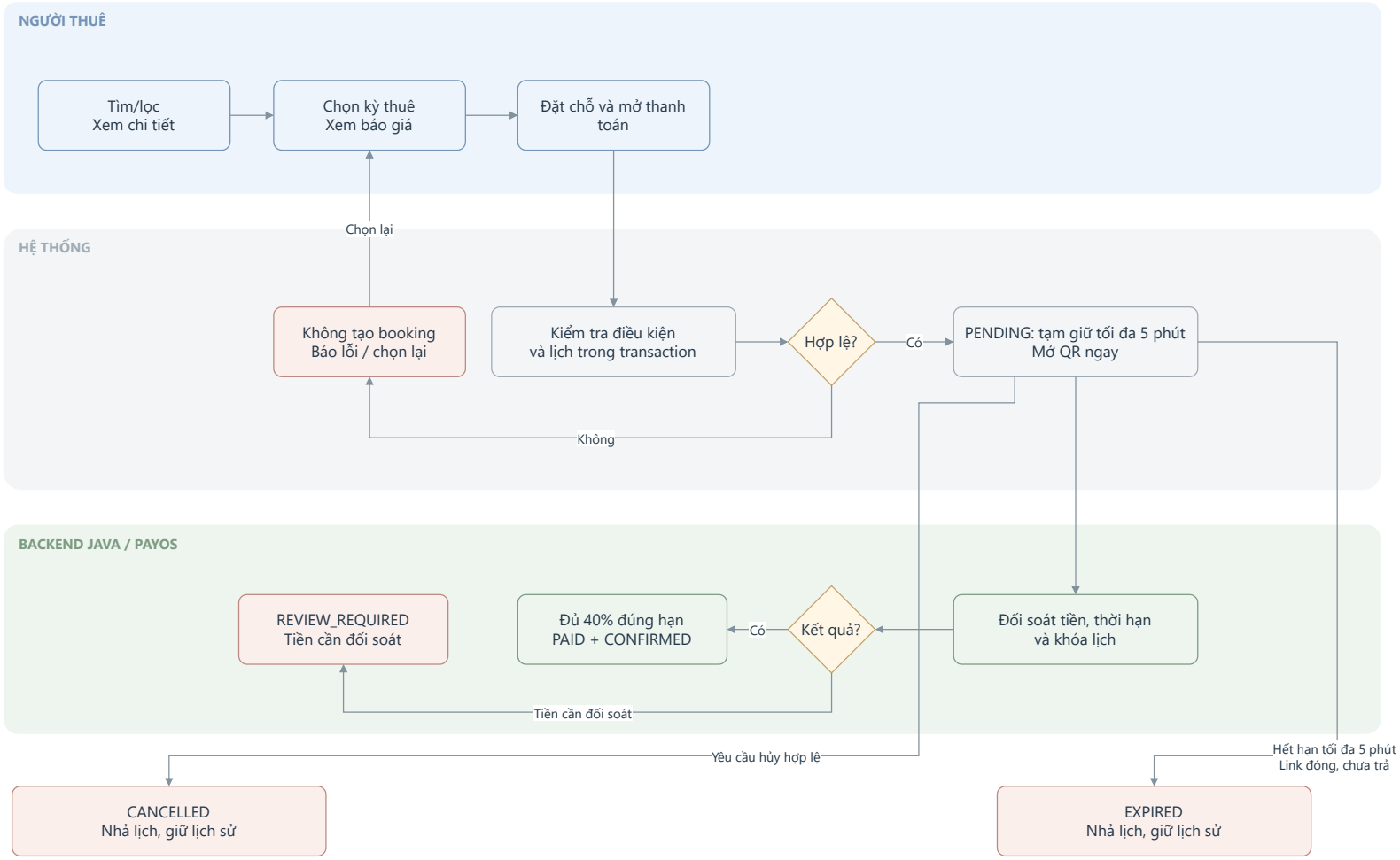
2.2.4 Thanh toán khoản trả trước

Người thuê thanh toán khoản trả trước 40% qua QR trong phạm vi một Host và kênh payOS demo đã cấu hình. Booking đang chờ thanh toán chưa được trình bày như một lượt đặt chỗ thành công. Backend Java kiểm tra danh tính, booking và giá đã chốt trước khi tạo hoặc lấy lại mã QR của cùng một yêu cầu thanh toán. Thời hạn thanh toán tính từ lúc tạo booking trên máy chủ, tối đa 5 phút và không vượt giờ bắt đầu thuê. Quay lại hoặc tiếp tục thanh toán dùng cùng mã đơn và cùng thời hạn; thao tác này không gia hạn giữ chỗ.

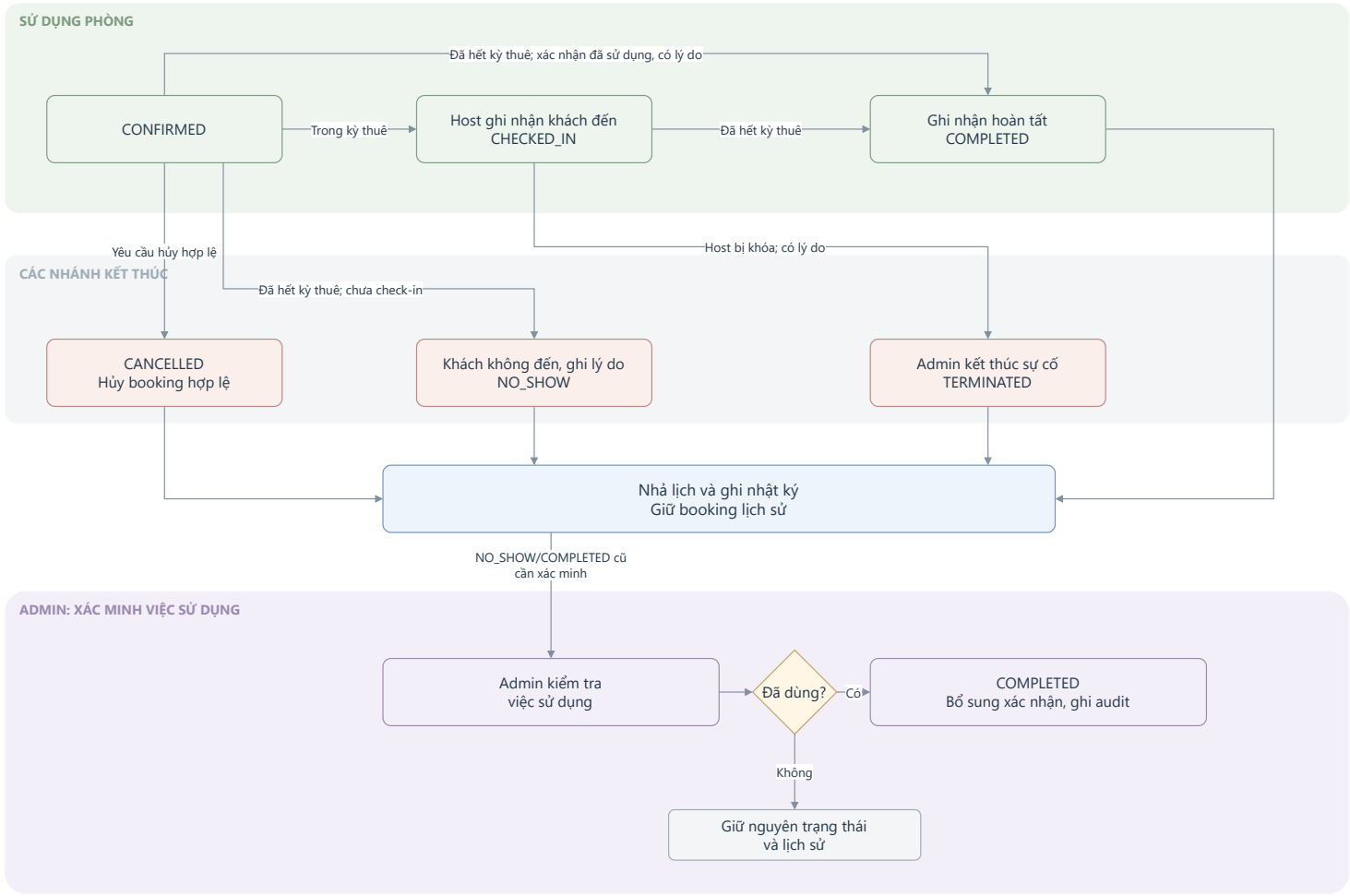
Kết quả do payOS gửi qua webhook chuyển tiếp được backend kiểm tra trước khi cập nhật dữ liệu. Khi nhận đủ tiền đúng hạn và khóa lịch còn hợp lệ, backend ghi `PAID` và xác nhận booking trong transaction. Ứng dụng nhận thay đổi để cập nhật QR, bộ đếm và thông báo; thanh toán thành công không tự ghi nhận khách đã đến nhận phòng. Giao dịch cần đối soát được tách khỏi trạng thái thanh toán hợp lệ, còn hoàn tiền tự động chưa thuộc phạm vi triển khai. Hình 2.2.4 thể hiện các thành phần tham gia luồng này.



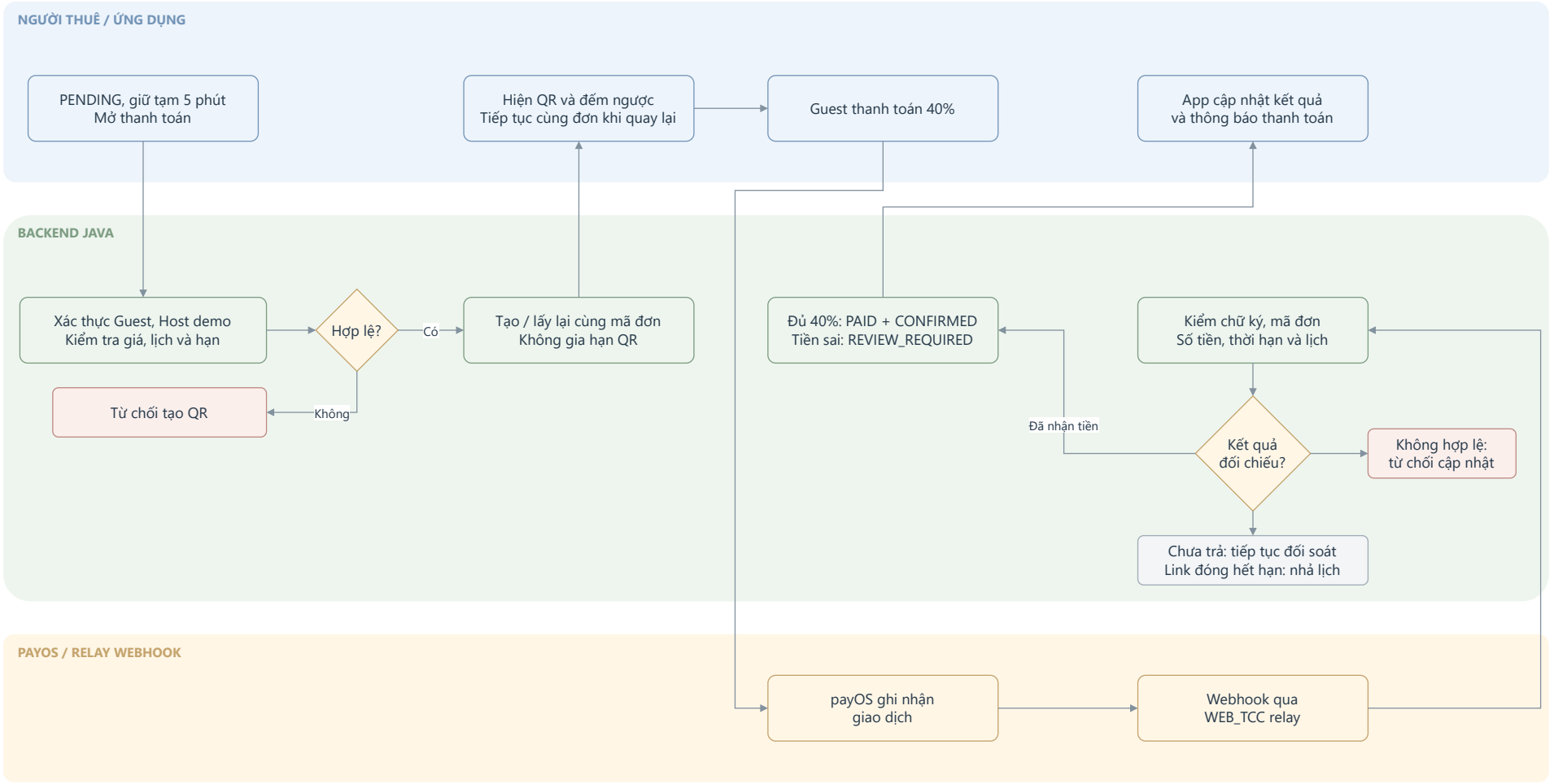
Hình 2.2.1: Quy trình cấp quyền Host và công bố không gian.



Hình 2.2.2: Quy trình tạo, giữ lịch và xử lý yêu cầu đặt chỗ.



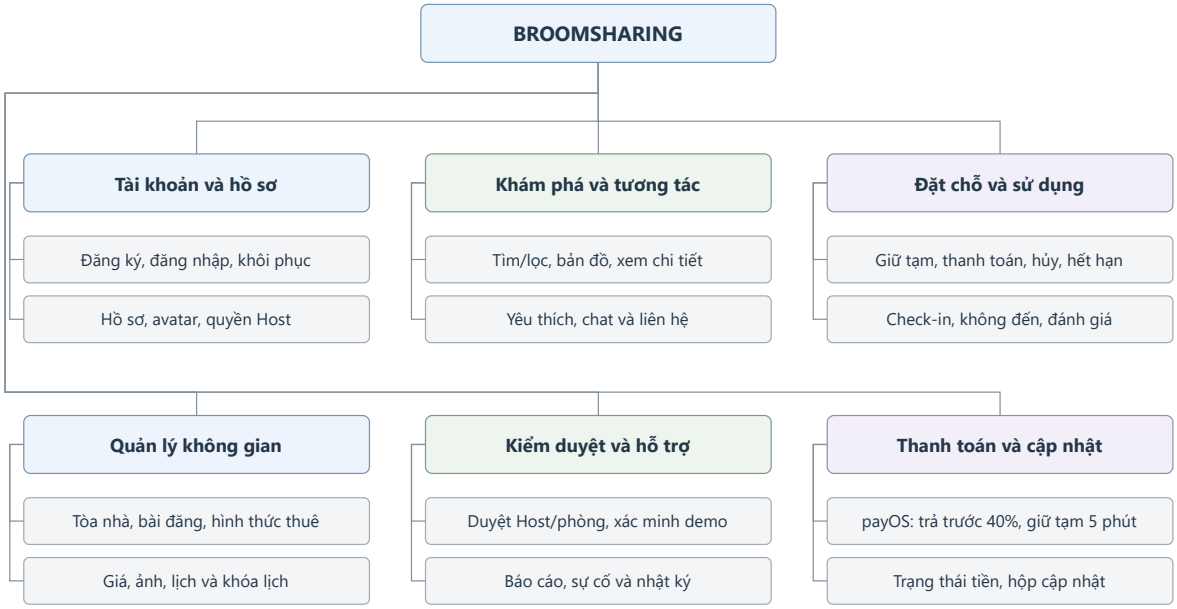
Hình 2.2.3: Quy trình sử dụng phòng, khép lượt thuê và xác minh ngoại lệ.



Hình 2.2.4: Luồng tạo QR và cập nhật kết quả thanh toán payOS.

2.3 Sơ đồ phân rã chức năng

Các chức năng được phân rã từ mục tiêu của từng vai trò đến nhóm thao tác liên quan. Nhóm người thuê bao gồm tài khoản, khám phá, booking, tương tác và hồ sơ Host; nhóm Host tập trung vào phòng, lịch và vận hành lượt thuê; nhóm Admin tập trung vào xét duyệt, xử lý vi phạm và kiểm tra nghiệp vụ. Cách phân rã tại Hình 2.3.1 làm rõ trách nhiệm mà không đồng nhất mỗi màn hình với một chức năng độc lập.



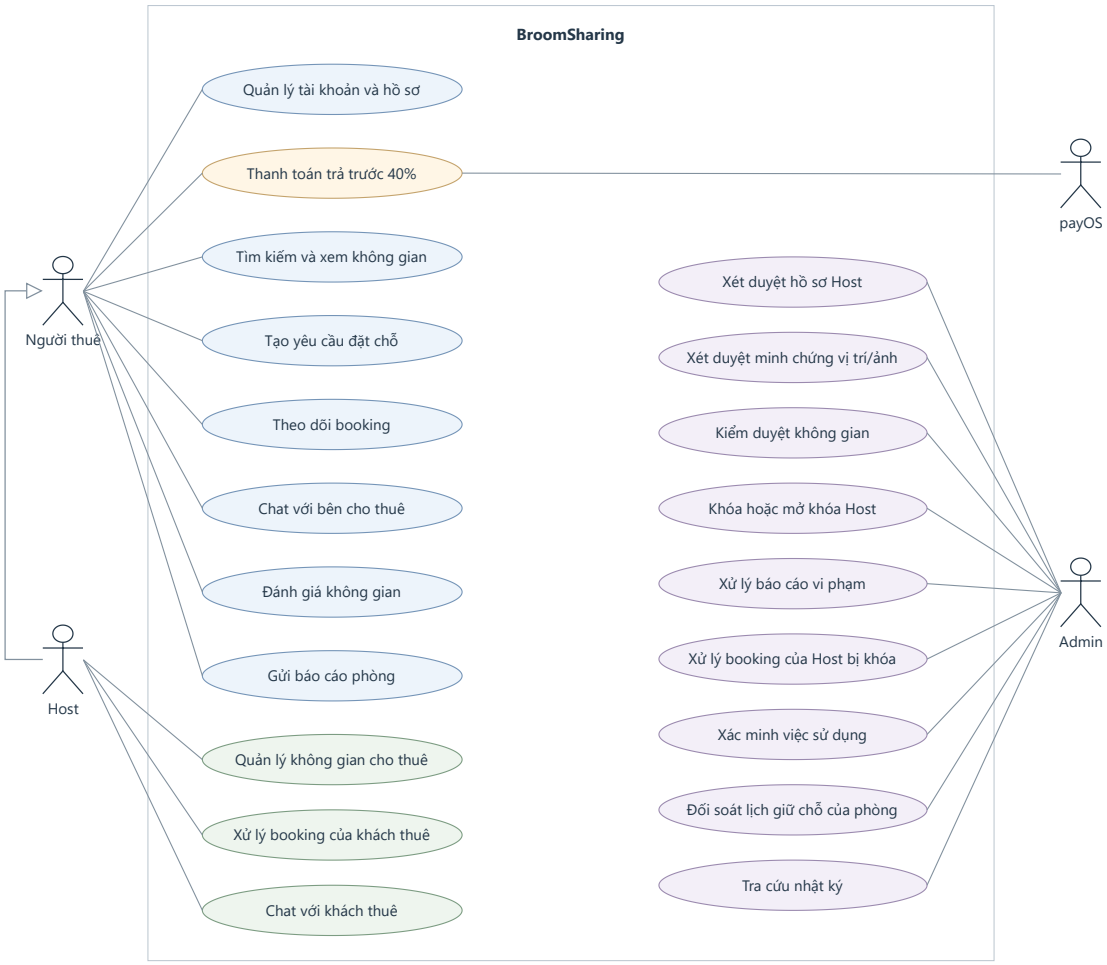
Hình 2.3.1: Phân rã chức năng tổng quát của BroomSharing.

Chức năng tin nhắn hỗ trợ trao đổi giữa hai bên của một hội thoại. Hộp cập nhật tổng hợp sự kiện booking và trạng thái đã đọc của từng tài khoản. Đánh giá gắn với lượt đã sử dụng, còn báo cáo vi phạm được chuyển đến Admin và cho phép Host phản hồi. Các chức năng này bổ sung thông tin xử lý cho lượt thuê, đồng thời duy trì giới hạn truy cập theo vai trò và chủ thể liên quan.

Các nhánh phân rã theo từng nhóm được trình bày tại Phụ lục G.

2.4 Sơ đồ use case

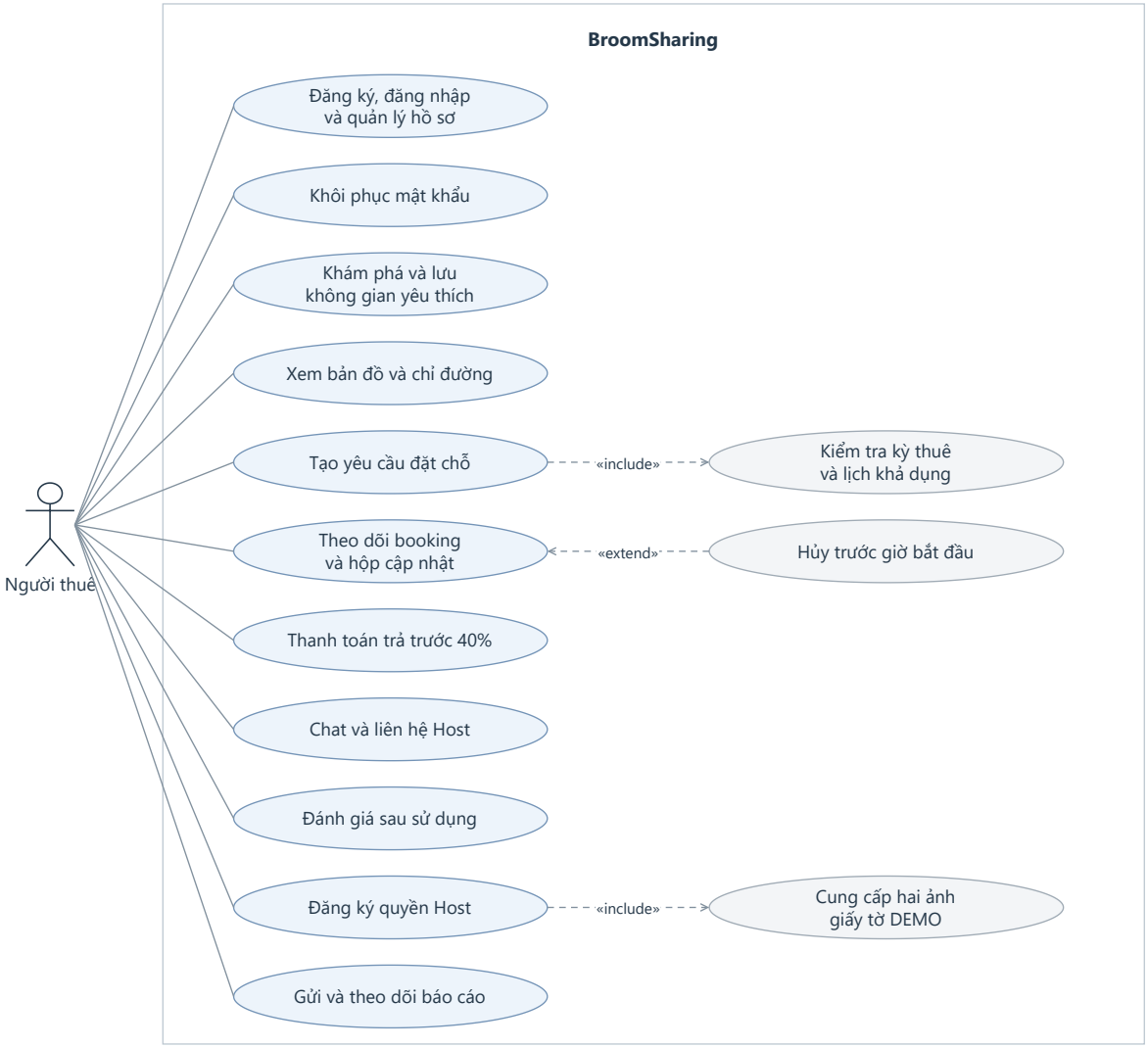
Hình 2.4.1 mô tả các mục tiêu tương tác với hệ thống. Host kế thừa khả năng thuê phòng của người thuê và có thêm nhóm chức năng cho thuê. Admin có tuyến chức năng kiểm duyệt riêng; quyền quản trị không mặc định bao gồm đọc hội thoại riêng của người dùng.



Hình 2.4.1: Use case tổng quát với các tác nhân người thuê, Host, Admin và dịch vụ thanh toán.

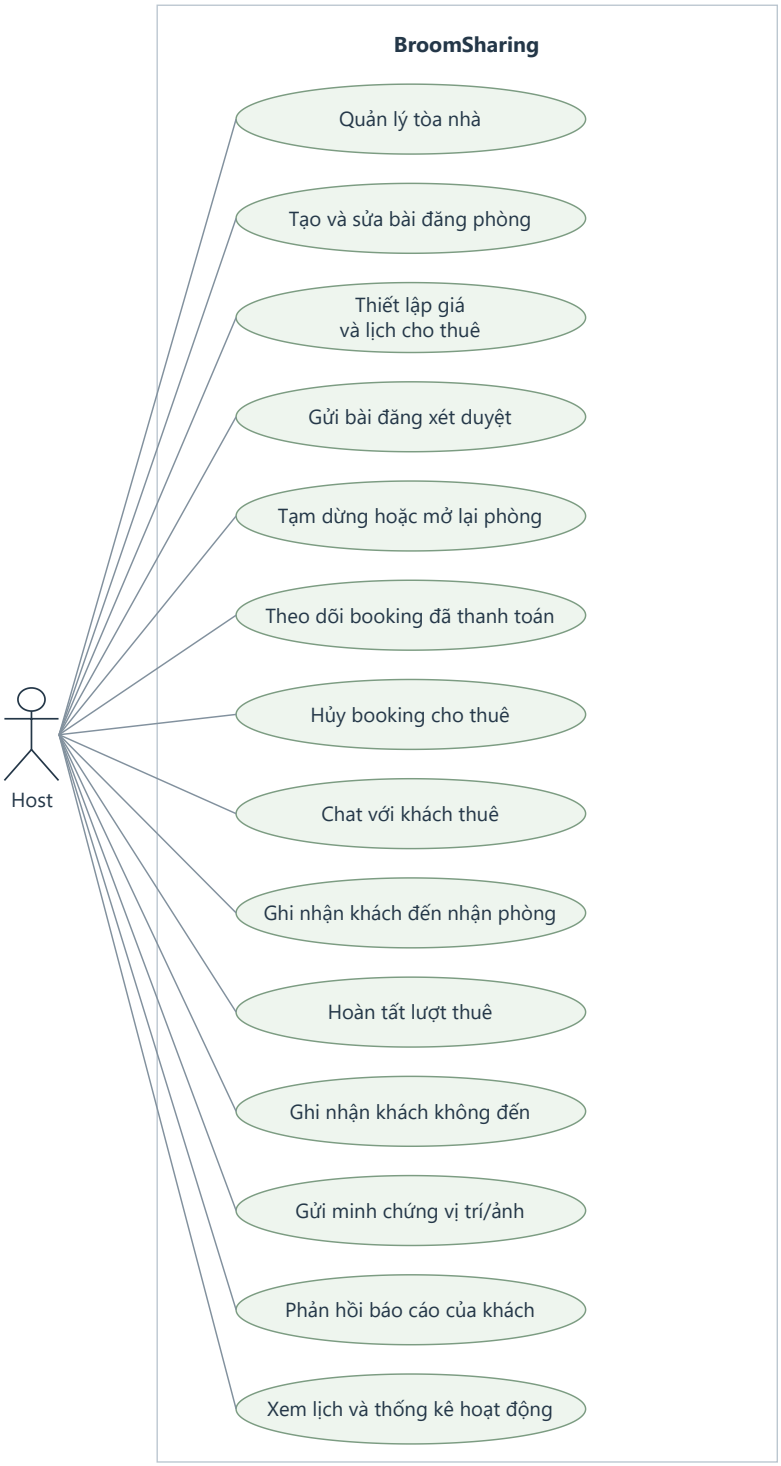
Ở phân hệ người thuê, đặt chỗ luôn bao gồm kiểm tra khả dụng và mở bước thanh toán, còn hủy booking là thao tác có điều kiện trên lượt đang được theo dõi. Gửi đơn làm Host bao gồm cung cấp hai ảnh DEMO. Các phân hệ Host và Admin giữ liên kết tác nhân với mục tiêu nghiệp vụ; những cập nhật lịch và nhật ký được mô tả trong quy trình xử lý. Hình 2.4.2, Hình 2.4.3 và Hình 2.4.4 trình bày chi tiết mục tiêu của từng phân hệ.

Phân hệ người thuê bao quát truy cập tài khoản, khám phá, đặt chỗ và tương tác sau thuê. Kiểm tra khả dụng và mở thanh toán thuộc thao tác đặt chỗ; hủy booking chỉ thực hiện khi lượt thuê còn đáp ứng điều kiện hủy.



Hình 2.4.2: Use case chi tiết của người thuê.

Host quản lý tòa nhà, phòng, giá và lịch, đồng thời ghi nhận việc sử dụng và các ngoại lệ của lượt thuê. Quyền thuê dùng chung được kế thừa tại sơ đồ tổng quát; trang này tập trung vào các mục tiêu cho thuê.



Hình 2.4.3: Use case chi tiết của Host.

Admin xét duyệt hồ sơ, bài đăng và minh chứng, xử lý vi phạm và can thiệp các booking thuộc phạm vi quản trị. Xác minh việc sử dụng và đối soát lịch được tách thành các mục tiêu riêng để phản ánh điều kiện xử lý khác nhau.



Hình 2.4.4: Use case chi tiết của Admin.

2.5 Đặc tả các nghiệp vụ tiêu biểu

2.5.1 Gửi yêu cầu đặt chỗ

Mục tiêu của nghiệp vụ là tạo lượt thuê có giá và quyền giữ lịch xác định. Điều kiện đầu vào gồm tài khoản được phép thuê, phòng đã duyệt và hoạt động, Host hợp lệ và thời gian chưa bắt đầu; không được đặt phòng của chính mình. Với thuê theo giờ, khoảng chọn phải đáp ứng ngày làm việc, giờ mở cửa và thời lượng tối thiểu. Cả hai chế độ đều được kiểm tra khoảng khóa liên quan.

Với thuê theo giờ, ứng dụng xác định các slot kỹ thuật 30 phút cần giữ trong khoảng được chọn; với thuê cố định, yêu cầu giữ toàn bộ một hoặc ba tháng. Hai chế độ không được dùng đồng thời cho cùng phòng. Mức giá dự kiến trên giao diện chỉ phục vụ xác nhận lựa chọn; trước khi tạo booking, dữ liệu phòng và lịch tiếp tục được kiểm tra trong transaction. Kết quả thành công là booking `PENDING` cùng các khóa lịch tương ứng, chưa phải booking đã thanh toán thành công. Màn hình tiếp theo mở QR; backend mới có quyền xác nhận kết quả nhận tiền.

Nhánh không thành công phân biệt dữ liệu nhập không phù hợp với lịch vừa bị người khác giữ. Nếu hai tài khoản gửi yêu cầu trùng nhau, chỉ yêu cầu còn thỏa điều kiện khi ghi dữ liệu được chấp nhận. Khi gửi lại cùng định danh yêu cầu và cùng nội dung, Repository trả về booking đã có thay vì tạo thêm bản ghi. Cách xử lý này tránh tạo lặp khi kết quả phản hồi chậm hoặc người dùng gửi lại thao tác.

2.5.2 Điều kiện xử lý và khép lượt thuê

Mỗi thao tác phải kiểm tra trạng thái hiện tại và quyền của người thực hiện, kể cả khi nút xử lý đã được hiển thị trước đó. Đối với hủy thông thường, booking phải còn ở trạng thái cho phép và chưa tới giờ bắt đầu; Host phải cung cấp lý do. Kết quả được lưu cùng người xử lý và mốc thời gian để các bên nhận biết nguồn gốc thay đổi.

Khi khép lượt, hệ thống giải phóng đúng lịch do booking đó giữ và bảo toàn lịch sử. Nếu Admin bổ sung xác nhận sử dụng hoặc sửa kết luận không đến, thao tác chỉ cập nhật kết quả liên quan, không nhả lịch lần thứ hai. Quyền đánh giá được kiểm tra từ người thuê, trạng thái đã khép và xác nhận sử dụng, thay vì chỉ dựa vào việc thời gian thuê đã kết thúc.

2.5.3 Báo cáo vi phạm và xử lý quản trị

Người thuê gửi báo cáo đối với phòng, mô tả vấn đề cần giải quyết; Host có thể phản hồi khi báo cáo còn mở. Admin xem thông tin đối tượng cùng phản hồi trước khi giải quyết hoặc bác báo cáo. Nếu cần hạn chế hoạt động, Admin có thể đình chỉ phòng hoặc khóa Host trong phạm vi thao tác được hỗ trợ. Báo cáo là hồ sơ xử lý vi phạm, không thay thế đánh giá của khách về một lượt sử dụng.

Đình chỉ phòng làm thay đổi khả năng nhận yêu cầu mới, nhưng không xóa lịch sử booking. Với Host bị khóa, các lượt chưa khép phải được xem xét từng trường hợp theo quy trình ngoại lệ. Việc gỡ đình chỉ cũng không tự mở phòng trở lại cho thuê: phòng chuyển về không hoạt động để Host chủ động kiểm tra trước khi công bố. Cách tách các quyết định này giúp tránh một thao tác quản trị làm thay đổi ngoài ý muốn toàn bộ vòng đời của các lượt thuê liên quan.

2.6 Yêu cầu phi chức năng và các ràng buộc thiết kế

Về tính nhất quán, một phòng chỉ được giữ độc quyền trong khoảng thuê, kể cả trong thời hạn tạm giữ chờ thanh toán. Các thay đổi booking, lịch và thông tin kiểm soát của phòng phải cùng thành công hoặc cùng bị từ chối trong thao tác tương ứng. Lịch không được chỉnh bằng cách chỉ ẩn một booking trên giao diện; các khóa đang giữ phải được giải phóng theo chính bản ghi đã tạo chúng.

Về phân quyền, ứng dụng kiểm tra cả vai trò và quan hệ với đối tượng. Host chỉ quản lý tài sản thuộc sở hữu; người thuê chỉ thực hiện quyền của mình trên booking; thành viên hội thoại mới được đọc và gửi tin trong hội thoại đó. Firestore Security Rules cho phép kiểm tra danh tính và dữ liệu trong điều kiện cho phép hoặc từ chối thao tác (Firebase, n.d.-j). Backend thanh toán sử dụng thông tin xác thực và kiểm tra nghiệp vụ phía máy chủ, không lấy số tiền do màn hình tự khai báo làm căn cứ thu tiền.

Về khả năng sử dụng, giao diện phân biệt đang tải, danh sách trống, lỗi và kết quả thành công. Khi dữ liệu thay đổi từ tài khoản khác, màn hình cần cập nhật trạng thái và kiểm tra lại trước thao tác quan trọng. Lỗi kết nối phải có phản hồi và cho phép thử lại khi phù hợp, đồng thời tránh tạo lặp dữ liệu. Các yêu cầu này được đánh giá qua kịch bản thao tác trên thiết bị và kết nối cụ thể.

3. THIẾT KẾ GIAO DIỆN

3.1 Tổng quan về thiết kế giao diện

Giao diện BroomSharing sử dụng xanh lục, xanh ngọc làm điểm nhấn trên nền trắng và xám nhẹ. Các màn hình thống nhất cách bố trí tiêu đề, trường nhập, thẻ thông tin và nút thao tác chính để làm nổi bật nội dung không gian.

Danh sách phòng sử dụng `RecyclerView` để tái sử dụng các View khi hiển thị dữ liệu (Android Developers, n.d.-a). Thanh điều hướng người thuê gồm Trang chủ, Bản đồ, Lượt thuê, Tin nhắn và Cá nhân; Host và Admin có khu vực quản lý riêng.

Bộ hình trong chương là prototype ban đầu do nhóm xây dựng trên Figma, thể hiện bố cục và hướng tương tác dự kiến. Các màn hình đặt trọng tâm vào trình tự sử dụng và cách nhóm thông tin; giao diện hiện thực cùng kết quả kiểm thử được trình bày ở các chương sau. Những lựa chọn xuất hiện trong prototype vì vậy được xem xét ở mức phương án thiết kế, không đồng nhất với toàn bộ chức năng đã tích hợp.

3.1.1 Phân cấp thông tin và hành động

Các màn hình tìm thuê được tổ chức theo mức độ cần thiết của thông tin. Trang chủ ưu tiên ảnh, loại không gian và giá để hỗ trợ so sánh; chi tiết phòng bổ sung điều kiện sử dụng và vị trí; bước xác nhận tập hợp thời gian, liên hệ và chi phí trước khi gửi yêu cầu. Dữ liệu được mở dần theo tiến trình, tránh đặt toàn bộ nội dung của phòng lên thẻ danh sách.

Hành động chính sử dụng màu nhấn và tên gắn với mục tiêu như đăng nhập, thuê ngay hoặc gửi yêu cầu. Những thao tác phụ như xem lại, thay đổi thông tin và quay về được đặt gần khối liên quan. Đối với hủy, từ chối hoặc khóa đối tượng, thiết kế cần thể hiện rõ đích tác động và bước xác nhận, vì các thao tác này làm thay đổi quyền sử dụng hoặc trạng thái xử lý.

3.1.2 Trạng thái giao diện và tính nhất quán

Thiết kế phân biệt đang tải, dữ liệu trống, dữ liệu không hợp lệ và kết quả gửi. Danh sách chưa tải xong không được thể hiện như không có kết quả; nút đã nhấn cũng chưa phải xác nhận lưu thành công. Khi gửi biểu mẫu, trạng thái xử lý giúp hạn chế thao tác lặp. Nếu có lỗi, thông báo gắn với nội dung cần sửa và giữ lại phần dữ liệu còn phù hợp. Các phản hồi này là yêu cầu cho bản hiện thực từ prototype.

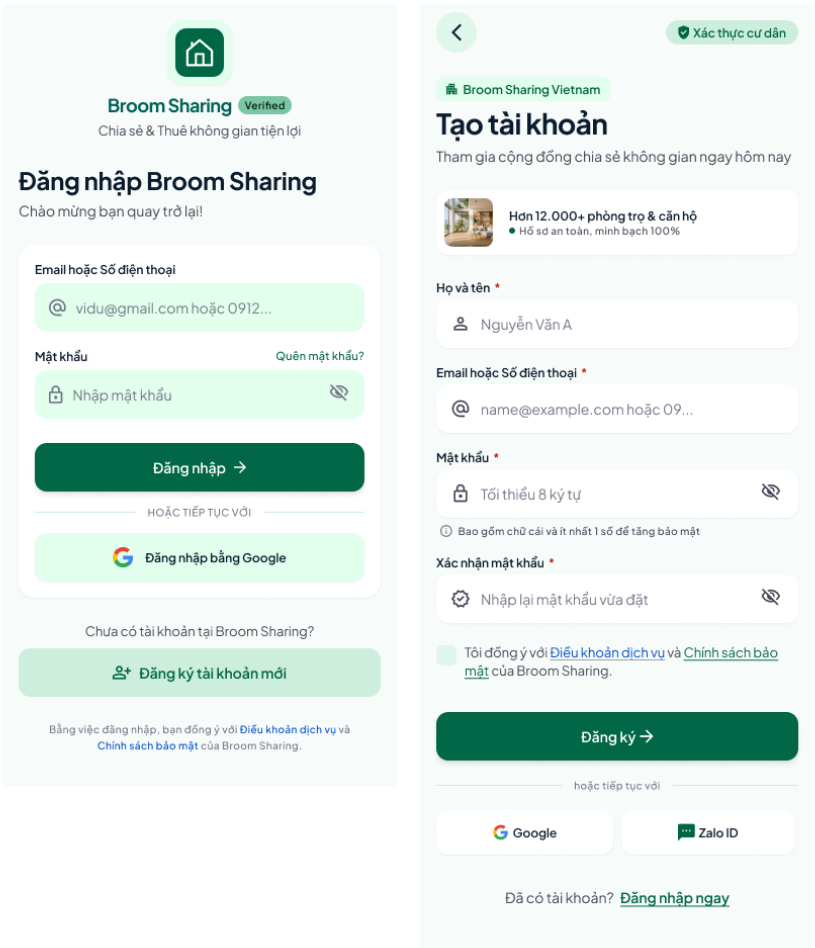
3.2 Giao diện phân hệ người thuê

3.2.1 Đăng nhập và đăng ký

Giao diện đăng nhập gồm tài khoản, mật khẩu và đường dẫn đến đăng ký, quên mật khẩu. Màn hình đăng ký thu thập họ tên, thông tin liên hệ, mật khẩu và xác nhận mật khẩu. Các trường sắp theo thứ tự nhập, với điều khoản sử dụng đặt trước nút hoàn tất.

Hình 3.2.1 phân nhóm trường nhập và làm nổi bật nút đăng nhập, đăng ký. Biểu tượng mật khẩu hỗ trợ kiểm tra nội dung nhập; lựa chọn Google/Zalo đặt ở vùng riêng để phân biệt với biểu mẫu tài khoản của ứng dụng.

Khi biểu mẫu có lỗi, phản hồi cần phân biệt thiếu dữ liệu, thông tin chưa đúng định dạng và xác thực không thành công. Nội dung mật khẩu được che mặc định; dữ liệu còn hợp lệ không bị xóa chỉ vì một trường khác cần sửa. Liên kết giữa đăng nhập và đăng ký giữ vai trò điều hướng, không làm người dùng nhầm với nút gửi biểu mẫu hiện tại.



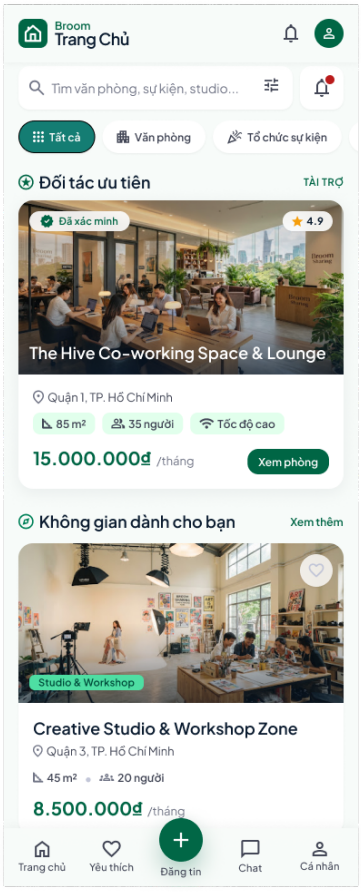
Hình 3.2.1: Thiết kế biểu mẫu đăng nhập và đăng ký tài khoản.

3.2.2 Trang chủ và chi tiết không gian

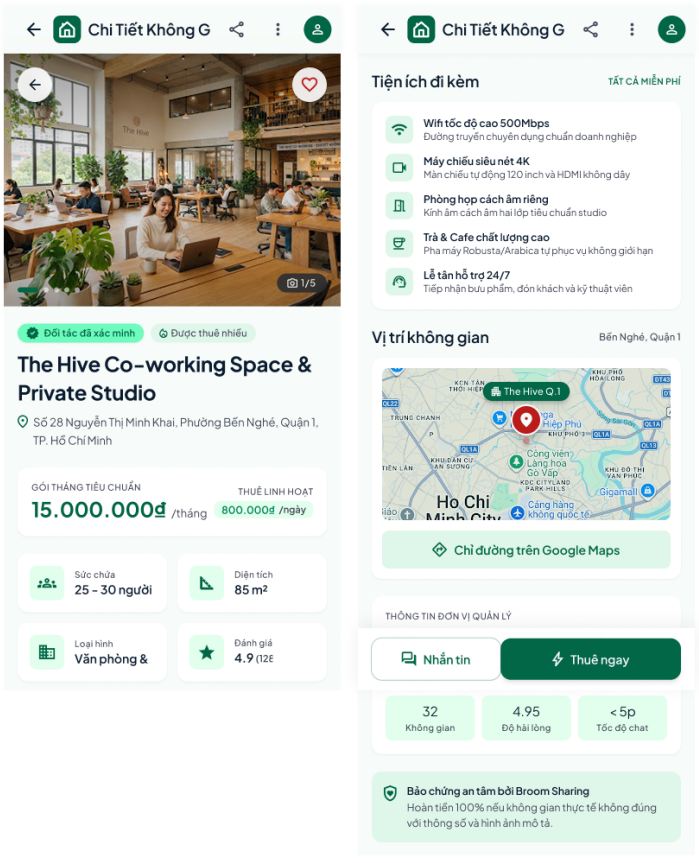
Giao diện Trang chủ tích hợp thanh tìm kiếm, phân loại danh mục và danh sách không gian dưới dạng thẻ. Mỗi thẻ tập trung vào ảnh, tên phòng, khu vực, sức chứa và giá thuê. Hình 3.2.2 minh họa thứ bậc hiển thị này qua hai vùng đối tác ưu tiên và không gian gợi ý. Thanh điều hướng đáy giữ các điểm truy cập chính ở vị trí ổn định khi duyệt danh sách.

Người dùng có thể tiếp cận bộ lọc từ vùng tìm kiếm và mở phòng cần xem. Màn hình chi tiết tại Hình 3.2.3 bổ sung bộ ảnh, diện tích, sức chứa, tiện ích, vị trí và thông tin Host. Nội dung dài được chia thành các khối để thuận tiện cuộn xem. Các hành động yêu thích, nhắn tin, chỉ đường và đặt thuê được bố trí gần thông tin liên quan để giảm thao tác chuyển màn hình.

Đơn vị giá cần đặt sát con số để phân biệt giá theo giờ với giá của kỳ thuê cố định. Các nhãn trạng thái và thông tin tiện ích giúp bổ sung ngữ cảnh, nhưng không thay thế phần điều kiện sử dụng trong chi tiết. Khi chuyển từ danh sách sang bản đồ hoặc mở phòng, lựa chọn phải tiếp tục trỏ đến cùng phòng để tránh nhầm đối tượng trước khi đặt.



Hình 3.2.2: Thiết kế Trang chủ.



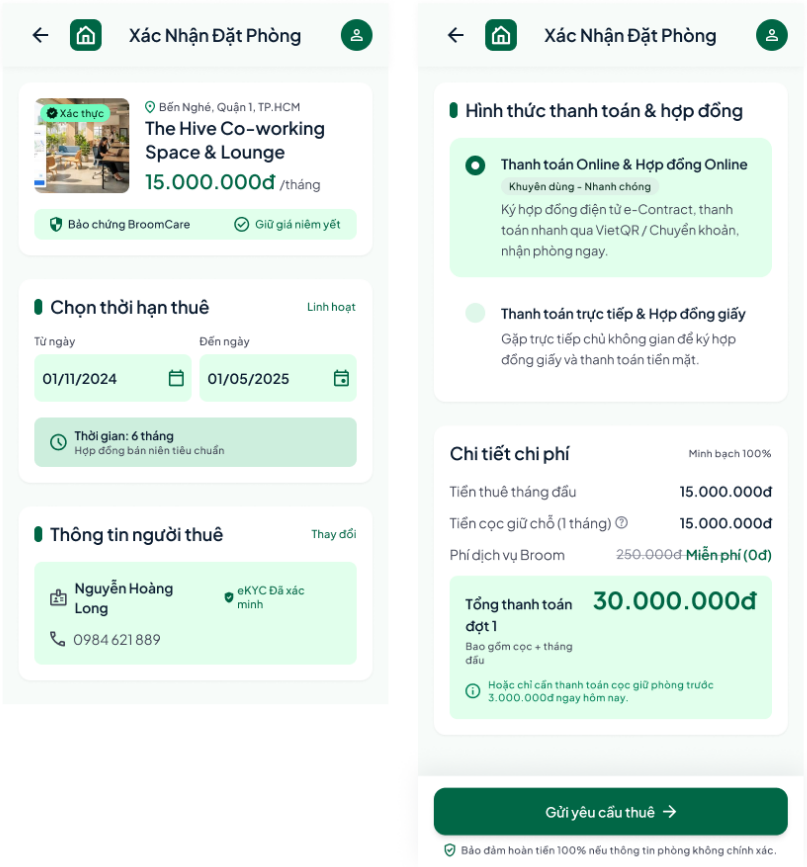
Hình 3.2.3: Thiết kế thông tin chi tiết không gian.

3.2.3 Đặt chỗ, theo dõi và thanh toán

Màn hình đặt chỗ tập hợp thời gian sử dụng, thông tin người thuê và giá dự kiến trước khi gửi yêu cầu. Khối thông tin phòng được giữ ở đầu màn hình để người dùng đối chiếu lựa chọn; vùng thời gian và liên hệ đặt ngay bên dưới. Đối với thiết kế nghiệp vụ, bộ chọn thời gian cần thích ứng với chế độ theo giờ hoặc kỳ thuê một, ba tháng của phòng.

Danh sách lượt thuê hiển thị thời gian, trạng thái booking và các thao tác còn hiệu lực. Phương án xác nhận tại Hình 3.2.4 tiếp tục chia nội dung thành hình thức thanh toán, thông tin hợp đồng và bảng chi phí. Tổng tiền cùng nút gửi yêu cầu nằm cuối tiến trình để người dùng kiểm tra trước khi xác nhận. Khi hiện thực luồng thanh toán, thông tin tiền cần tách khỏi trạng thái nhận phòng và hoàn tất.

Ở bước xác nhận, thay đổi kỳ thuê phải dẫn đến cập nhật phần thời gian và chi phí liên quan. Thiết kế dành vị trí cho thông tin người thuê, cho phép đối chiếu liên hệ mà không phải quay về hồ sơ để xem lại. Các nhãn bảo chứng, lựa chọn hợp đồng và số tiền cụ thể trong hình là nội dung minh họa của phương án ban đầu; thiết kế chi tiết khi triển khai phải bám chính sách đã xác định ở Chương 2.



Hình 3.2.4: Prototype xác nhận thông tin và chi phí đặt phòng.

3.2.4 Hồ sơ và tương tác

Giao diện Cá nhân phân nhóm hồ sơ, ảnh đại diện, phòng yêu thích, lịch sử và đăng ký quyền Host. Khu vực tin nhắn tổ chức hội thoại với chủ không gian; hộp cập nhật tập trung sự kiện liên quan đến lượt thuê và trạng thái đã đọc.

Biểu mẫu đánh giá gồm điểm và nhận xét, chỉ mở sau lượt thuê đã xác nhận sử dụng và khép lại. Báo cáo vi phạm có nội dung riêng gửi Admin, tách khỏi nhận xét công khai về phòng.

Đối với hội thoại, danh sách cần thể hiện người trao đổi, nội dung gần nhất và tình trạng đã đọc; màn hình chat giữ ngữ cảnh phòng để hỗ trợ trao đổi. Thông tin chưa đọc phục vụ việc nhận biết thay đổi, không làm phát sinh một trạng thái mới của booking. Các đường dẫn từ thông báo hoặc hội thoại phải mở đúng đối tượng liên quan thay vì chỉ đưa về Trang chủ.

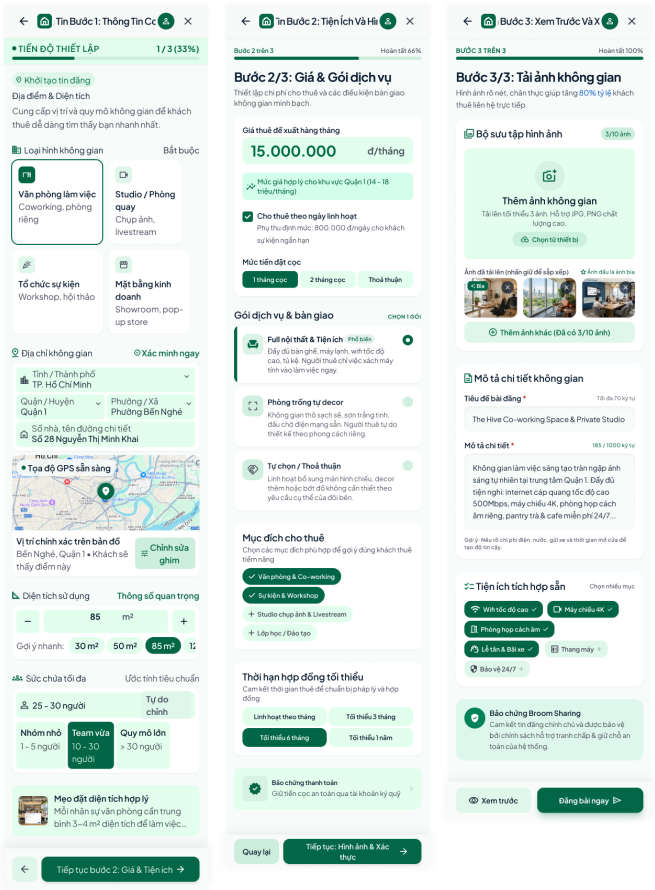
3.3 Giao diện phân hệ chủ không gian

Giao diện Host tập trung vào phòng, booking và lịch hoạt động. Khi đăng phòng, biểu mẫu phân nhóm thông tin cơ bản, giá và lịch, hình ảnh và điều kiện sử dụng. Hình 3.3.1 minh họa cách giữ các trường cùng mục đích gần nhau trong ba bước nhập liệu.

Thông tin cơ bản gồm vị trí, loại không gian, diện tích và sức chứa. Phần giá và lịch tuân theo chế độ thuê của phòng; phần hình ảnh tập trung ảnh, mô tả và tiện ích. Thanh tiến trình cùng nút tiếp tục, quay lại giúp nhận biết bước hiện tại và kiểm tra thông tin trước khi gửi bài.

Tại màn hình booking, các thao tác của Host được phân biệt theo trạng thái: xác nhận, từ chối, hủy, nhận phòng, hoàn tất hoặc ghi nhận khách không đến. Những thao tác cần giải trình có ô nhập lý do. Màn hình lịch và danh sách phòng bổ sung góc nhìn tổng hợp về khoảng đã giữ, trạng thái bài đăng và yêu cầu cần xử lý.

Khi đăng tin theo nhiều bước, thông tin đã nhập cần được giữ trong quá trình đi tới hoặc quay lại giữa các bước của biểu mẫu. Vùng ảnh cần phân biệt ảnh đã chọn, đang tải và tải lỗi trước khi gửi bài. Đối với quản lý phòng, trạng thái chờ duyệt và trạng thái tạm ngừng không được thể hiện bằng cùng một nhãn, vì chúng có nguyên nhân và hành động tiếp theo khác nhau.



Hình 3.3.1: Prototype ba bước nhập thông tin và đăng không gian.

3.4 Giao diện phân hệ quản trị viên

Giao diện Admin phân chia khu vực tổng quan, xét duyệt, quản lý và báo cáo. Hồ sơ, bài đăng được hiển thị cùng trạng thái và mở chi tiết trước khi chấp thuận hoặc từ chối. Đơn xin quyền Host và minh chứng vị trí/hình ảnh là các nhóm độc lập.

Hình A.1 đặt bộ lọc phía trên danh sách, gắn nút phê duyệt hoặc từ chối với hồ sơ đang xem. Trạng thái và lý do xử lý phân biệt khóa Host, đình chỉ phòng và khép booking. Các con số trong prototype phục vụ minh họa bố cục.

Trong thiết kế, danh sách cung cấp thông tin nhận diện nhanh, còn quyết định xử lý gắn với hồ sơ cụ thể. Trường hợp từ chối hoặc áp dụng hạn chế cần có vùng nhập lý do và xác nhận hành động. Hai nhóm kiểm duyệt phòng và quản lý tài khoản được bố trí riêng để phân biệt đối tượng chịu tác động; prototype đầy đủ được trình bày tại Phụ lục A.

4. THIẾT KẾ CƠ SỞ DỮ LIỆU

BroomSharing sử dụng database `broomsharing` của Cloud Firestore để lưu dữ liệu tài khoản, không gian, đặt chỗ và trao đổi giữa các bên. Firebase Authentication quản lý danh tính đăng nhập, liên kết với hồ sơ người dùng qua UID. Cloudinary lưu ảnh và minh chứng DEMO, còn Firestore giữ URL tham chiếu. Backend Java quản lý giao dịch payOS và trạng thái tiền (Nhóm phát triển BroomSharing, n.d.).

4.1 Các thực thể và thuộc tính

Dữ liệu ứng dụng được tổ chức thành 17 collection gốc và ba loại subcollection, tương ứng 20 mẫu document đang được ứng dụng hoặc backend sử dụng trong Bảng 4.1.1. Các thuộc tính định danh liên kết tài khoản với tài sản và yêu cầu thuê; nhóm trạng thái chi phối việc công bố phòng, xét duyệt và chuyển bước đặt chỗ. Collection thông báo cũ chỉ còn contract trong Rules, không phải nguồn hộp cập nhật đang hoạt động. Document còn chứa dữ liệu nhúng dạng `map`, `array` và các mốc thời gian xử lý. Những trường như người duyệt hoặc thời điểm hủy chỉ phát sinh khi nghiệp vụ tương ứng được thực hiện.

Bảng 4.1.1: Các mẫu document và thuộc tính tiêu biểu

Đường dẫn	Vai trò	Thuộc tính tiêu biểu và kiểu
`users/{uid}`	Hồ sơ và quyền tài khoản.	`uid`, `role`, `status`, `photoUrl` (`string`); ...
`publicHosts/{uid}`	Bản chiếu Host.	`uid`, `name`, `photoUrl` (`string`); ...
`usernames/{normalizedUsername}`	Ánh xạ đăng nhập.	`uid`, `email` (`string`).
`hostApplications/{uid}`	Đơn cấp quyền Host.	`applicantUid`, `status` (`string`); `identityImageUrls` (`array<string>`); ...
`users/{uid}/favorites/{roomId}`	Phòng yêu thích.	`roomId`, `roomName` (`string`); `createdAt` (`timestamp`).
`users/{uid}/noticeReads/{eventId}`	Mốc đọc sự kiện dẫn xuất.	`readAt` (`timestamp`).
`buildings/{buildingId}`	Địa điểm và chủ sở hữu.	`ownerUid`, `name`, `address` (`string`); `location` (`geopoint/null`); ...
`rooms/{roomId}`	Không gian và trạng thái tin.	`buildingId`, `ownerUid`, `activeMode` (`string`); `activeBookingCount` (`integer`); ...
`bookings/{bookingId}`	Kỳ thuê, trạng thái và giá đã chốt.	`roomId`, `guestUid`, `status`, `paymentFlow` (`string`); `priceSnapshot` (`map`); ...

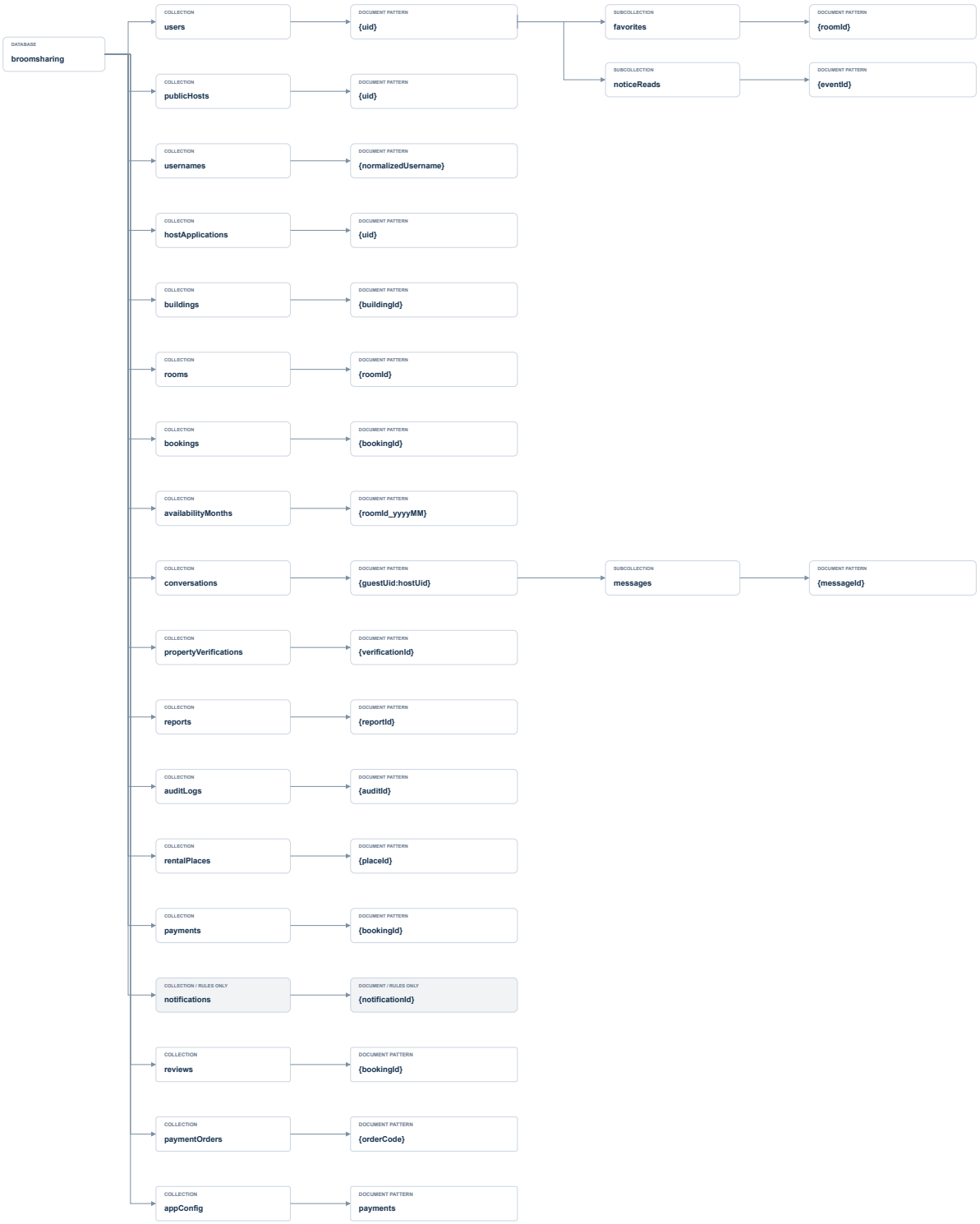
Còn tiếp ở trang sau

Đường dẫn	Vai trò	Thuộc tính tiêu biểu và kiểu
availability Months/{roomId_ yyyyMM}	Khóa lịch chính thức và tạm giữ.	fixedBookingId (string); slotsBusy (array<integer>); hourlyBookings (map<string, array<integer>>); pendingPaymentIds (array<string>); ...
payments/{booking Id}	Giao dịch trả trước.	orderCode, amount, expiresAt (integer); status (string); ...
paymentOrders/{ orderCode}	Ánh xạ mã đơn của backend.	bookingId (string).
appConfig/ payments	Cấu hình kênh demo công khai.	flow, demoHostUid (string).
reviews/{booking Id}	Đánh giá sau sử dụng.	roomId, guestUid, comment (string); rating (integer); ...
conversations/{ guestUid:hostUid}	Hội thoại và mốc đọc.	memberUids (array<string>); sequence, guestRead, hostRead (integer); ...
conversations/{ guestUid:hostUid} /messages/{ messageId}	Tin nhắn của hội thoại.	senderUid, text (string); sequence (integer); createdAt (timestamp); ...
property Verifications/{ verificationId}	Minh chứng xác minh.	ownerUid, verificationType, status (string); evidenceRefs (array<string>); ...
reports/{report Id}	Phản ánh và kết quả xử lý.	reporterUid, roomId, bookingId, status (string); ...
auditLogs/{audit Id}	Dấu vết thay đổi.	actorUid, targetPath, action (string); before, after (map<string,any>); ...
rentalPlaces/{ placeId}	Địa điểm tham khảo, không đặt thuê.	name, sourceUrl (string); location (geopoint); ...

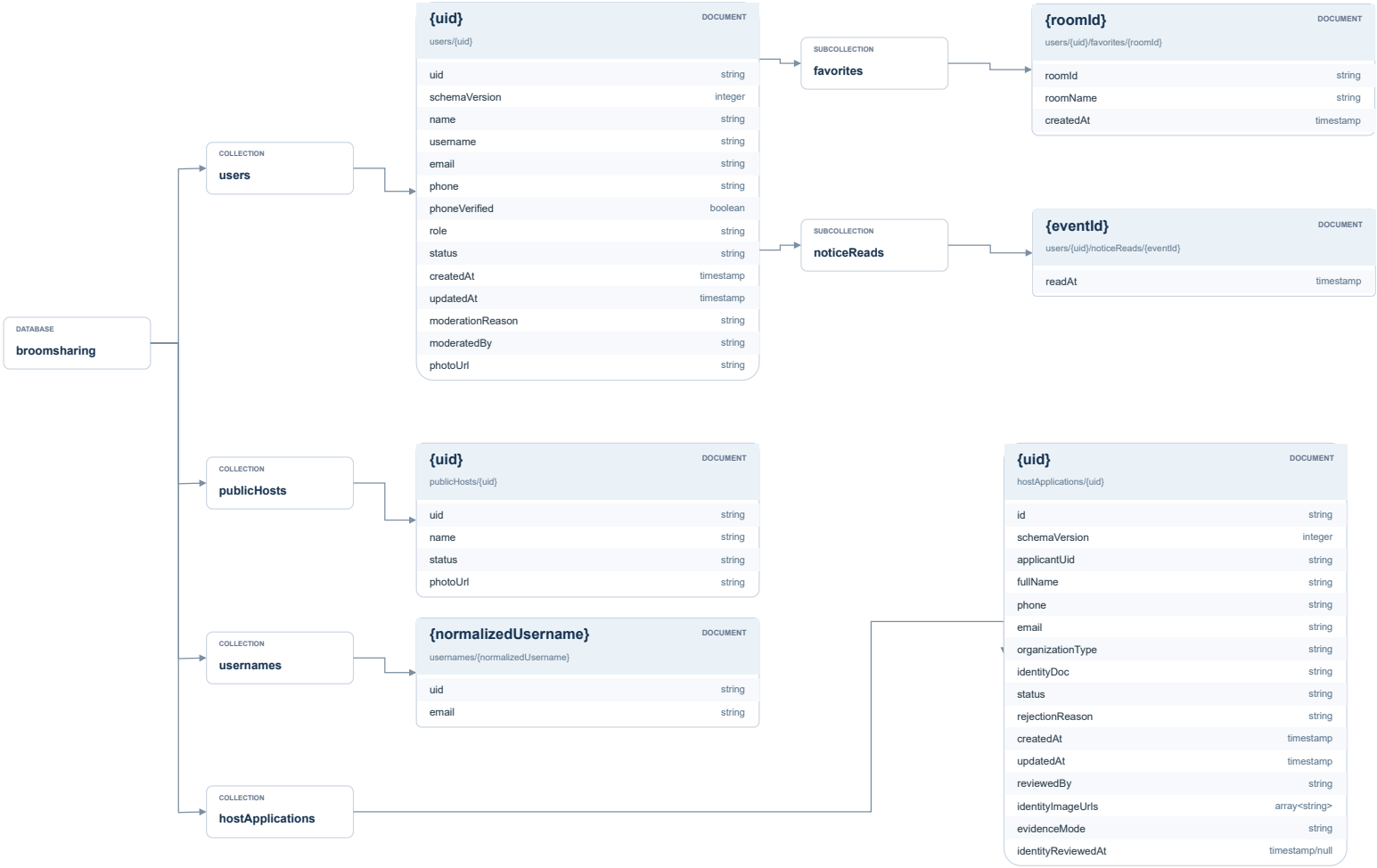
Mô hình tách hồ sơ tài khoản khỏi bản chiếu Host, thông tin phòng khỏi lịch giữ chỗ và nội dung hội thoại khỏi từng tin nhắn. Các lớp tổng hợp phục vụ giao diện như RoomListing, BookingNotice và HostStats không có collection độc lập. Trong booking, tên phòng, liên hệ người thuê và giá được lưu tại thời điểm đặt, duy trì ngữ cảnh của yêu cầu khi dữ liệu gốc thay đổi.

4.2 Quan hệ dữ liệu và cấu trúc lưu trữ

Hình 4.2.1 thể hiện cây lưu trữ của hệ thống. Hai subcollection favorites, noticeReads thuộc từng document người dùng; messages thuộc từng document hội thoại. Các collection buildings, rooms, bookings và availabilityMonths nằm ngang cấp, liên kết bằng các trường ID dạng chuỗi. Cấu trúc phân cấp này tuân theo mô hình collection và document của Firestore (Firebase, n.d.-b).



Hình 4.2.1: Cây lưu trữ Firestore BroomSharing. Thẻ màu xám là contract thông báo cũ trong Rules.



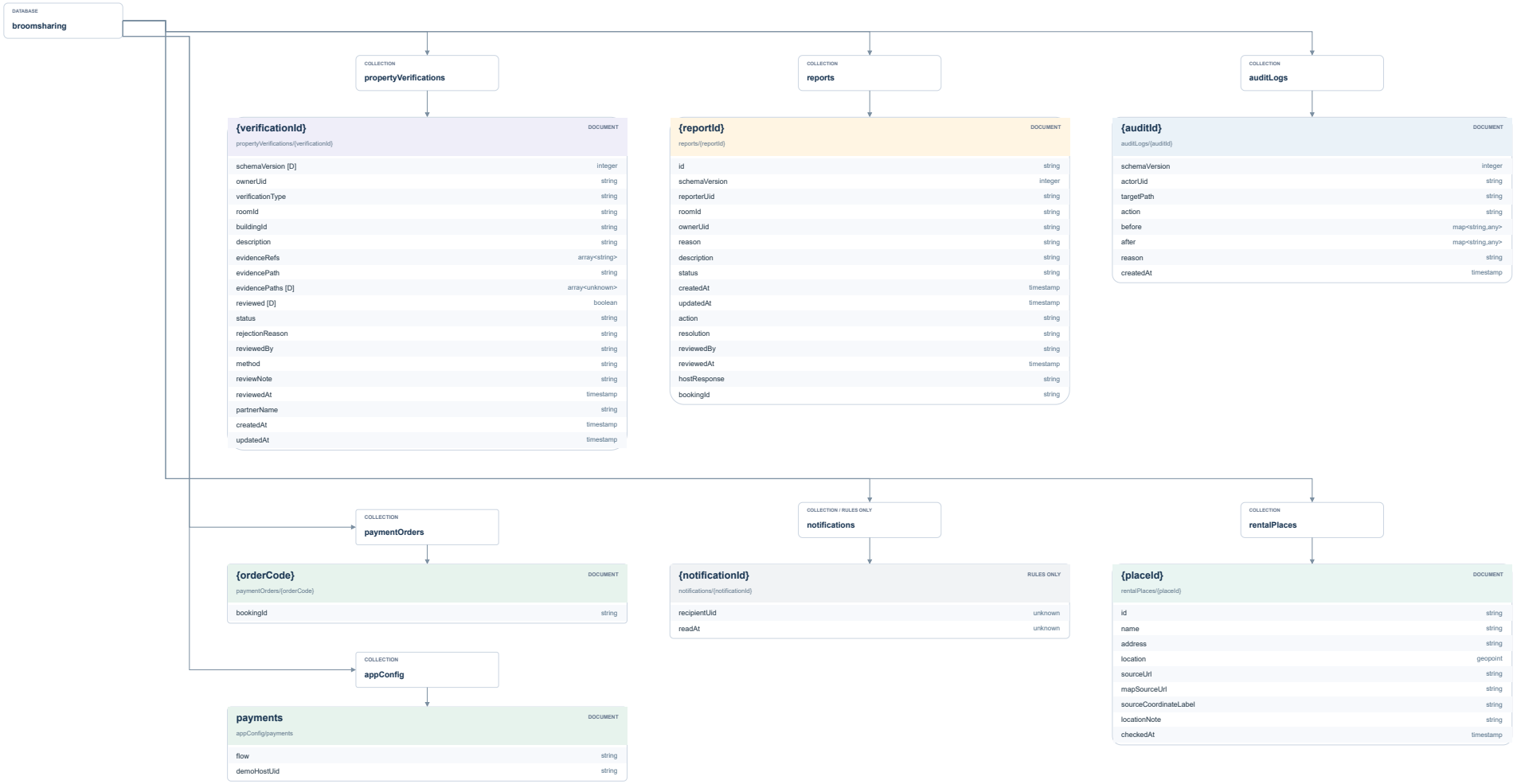
Hình 4.2.2: Cấu trúc nhóm tài khoản và các subcollection thuộc hồ sơ người dùng.



Hình 4.2.3: Cấu trúc tài sản, booking, lịch tháng, thanh toán và đánh giá. Các collection nằm ngang cấp trong database.



Hình 4.2.4: Cấu trúc hội thoại và subcollection tin nhắn của từng hội thoại.



Hình 4.2.5: Nhóm quản trị, dữ liệu tham khảo và cấu hình backend. [D] là trường từ dữ liệu mẫu; Rules-only là contract chưa có luồng ghi hiện tại.

Nhóm tài khoản trong Hình 4.2.2 dùng chung UID để liên kết hồ sơ, bản chiếu Host và đơn đăng ký. Nhóm đặt chỗ trong Hình 4.2.3 liên kết tài sản với booking và lịch tháng bằng ID. Trong nhóm chat, mỗi document hội thoại sở hữu một subcollection tin nhắn riêng (Hình 4.2.4); dữ liệu xác minh, báo cáo và nhật ký lưu thêm ID hoặc đường dẫn của đối tượng được xử lý (Hình 4.2.5). Các liên kết logic này có ý nghĩa và bội số như Bảng 4.2.1.

Bảng 4.2.1: Các quan hệ logic chính

Nhóm dữ liệu	Liên kết	Ý nghĩa và bội số
Tài khoản	UID của hồ sơ, alias và bản chiếu Host.	Một User có một alias theo luồng đăng ký và tối đa một bản chiếu Host; UID liên kết với Authentication.
Tài sản	`rooms.buildingId`; owner UID của Room và Building.	Một Building có nhiều Room; mỗi Room gắn với một Building và Host sở hữu tương ứng.
Đặt chỗ	Room ID, guest UID, owner UID.	Mỗi Booking thuộc một Room và hai bên tham gia; một Room có nhiều booking qua thời gian.
Lịch tháng	Room ID; fixed booking ID; khóa booking trong `hourlyBookings`.	Mỗi Room có nhiều tháng lịch; một tháng giữ toàn bộ cho một booking cố định hoặc giữ các slot thuê giờ.
Yêu thích	Room ID trong nhánh của User.	Quan hệ nhiều với nhiều giữa User và Room thông qua Favorite.
Hội thoại	`memberUids`; Room ID, Booking ID; sender UID của message.	Hai thành viên trong một hội thoại; một hội thoại có nhiều message và ngữ cảnh phòng hoặc booking.
Quản trị	UID chủ hồ sơ, người báo cáo, người duyệt; Room ID hoặc Building ID.	Hồ sơ xác minh và báo cáo liên kết tới đối tượng được xử lý; liên kết phụ thuộc loại hồ sơ.
Thanh toán	Booking ID của Payment; booking ID trong PaymentOrder.	Mỗi booking dùng một Payment cùng ID; order code ánh xạ về booking để đối soát webhook. Payment không phải subcollection của Booking.
Nhật ký	Actor UID và `targetPath`.	Actor xác định tài khoản hoặc tiến trình backend; đường dẫn đích thay đổi theo hành động.

Map `priceSnapshot` giữ giá và chính sách của booking; mảng `blackoutRanges` chứa các khoảng khóa lịch; map `hourlyBookings` ánh xạ booking ID tới tập slot. Đây là dữ liệu nhúng trong document. Room được thuê độc quyền trong khoảng $[startAt, endAt)$: PENDING giữ lịch tạm chờ thanh toán trong tối đa 5 phút, HOURLY dùng slot 30 phút và FIXED giữ trọn một hoặc ba tháng. Mảng `pendingPaymentIds` phân biệt các khóa tạm chờ thanh toán với lịch đã xác nhận. Transaction duy trì sự nhất quán giữa booking, lịch tháng và revision của phòng.

4.3 Một số thao tác dữ liệu tiêu biểu

4.3.1 Các nhóm thao tác dữ liệu

Các Repository thực hiện ba nhóm thao tác: đọc dữ liệu theo điều kiện, theo dõi thay đổi bằng listener và ghi dữ liệu nghiệp vụ. Truy vấn danh sách giới hạn phạm vi theo trạng thái hoặc UID; listener cập nhật booking và trao đổi giữa hai bên; transaction xử lý những thay đổi liên quan nhiều document. Bảng 4.3.1 tổng hợp các thao tác tiêu biểu và dữ liệu tham gia.

Bảng 4.3.1: Các thao tác dữ liệu tiêu biểu trong ứng dụng

Thao tác	Dữ liệu	Cách xử lý trong Repository
Tải danh sách phòng	`rooms`; bổ sung Building và PublicHost.	Lọc APPROVED, ACTIVE; sắp theo document ID, lấy tối đa 20 phòng và phân trang bằng con trỏ.
Theo dõi booking	`bookings`.	Lọc `guestUid` hoặc `ownerUid` theo tài khoản; nhận thay đổi qua listener.
Đọc lịch phòng	`availabilityMonths`.	Đọc các document tháng liên quan để xác định lịch giữ chỗ theo giờ hoặc theo tháng.
Tạo booking	Booking, lịch tháng, Room và AuditLog.	Đọc và kiểm tra trong transaction; ghi booking PENDING cùng khóa lịch, revision phòng và nhật ký.
Chuyển trạng thái booking	Booking, lịch tháng, Room và AuditLog.	Kiểm tra bên thực hiện, trạng thái và thời gian; hủy, hết hạn hoặc khép lượt giải phóng khóa lịch tương ứng; booking mới được backend xác nhận sau thanh toán.
Đối soát thanh toán	Payment, Booking, lịch tháng, Room và AuditLog.	Backend kiểm tra kết quả payOS; xác nhận khi nhận đủ tiền đúng hạn hoặc khép link chưa trả và nhả lịch, không tự hoàn tiền.
Theo dõi chat	`conversations` và subcollection `messages`.	Inbox lọc UID trong `memberUids`; tin nhắn sắp theo `sequence`, giới hạn số lượng và nhận cập nhật qua listener.
Gửi tin nhắn	Message và document hội thoại cha.	Transaction ghi tin, cập nhật preview, thứ tự và bộ đếm nhận tin của bên còn lại.
Lưu phòng yêu thích	`users/{uid}/favorites/{roomId}`.	Ghi ID, tên phòng và thời điểm lưu; bỏ yêu thích bằng cách xóa document tại cùng đường dẫn.

Thao tác đọc danh sách và theo dõi thay đổi phục vụ hai nhu cầu khác nhau. Danh sách phòng được tải theo từng trang, còn booking và chat cần phản ánh cập nhật trong quá trình sử dụng. Các listener này bỏ qua snapshot từ cache hoặc còn ghi đang chờ xác nhận, thông qua metadata `isFromCache()` và `hasPendingWrites()` (Firebase, n.d.-d).

4.3.2 Các truy vấn tiêu biểu

Khi tải danh sách không gian, `RoomRepository.page()` chỉ chọn phòng đã duyệt APPROVED và đang hiển thị ACTIVE. Tham số `after` là document ID cuối của lượt tải trước; ở lượt đầu, tham số này bằng `null`. Truy vấn sắp xếp theo document ID và giới hạn bằng hằng `PAGE_SIZE=20`; `startAfter()` đặt điểm bắt đầu cho lượt tiếp theo (Firebase, n.d.-g, n.d.-h).

```
Query query =
        db.collection("rooms")
                .whereEqualTo("approvalStatus", "APPROVED")
                .whereEqualTo("listingStatus", "ACTIVE")
                .orderBy(FieldPath.documentId())
                .limit(PAGE_SIZE);
if (after != null) query = query.startAfter(after);
query.get(Source.SERVER)
```

Nguồn: `RoomRepository.java`, dòng 63 đến 70.

Kết quả được đọc từ máy chủ bằng `get(Source.SERVER)`. Repository lấy ID của document cuối làm con trỏ, rồi đọc bổ sung Building và PublicHost để tạo danh sách `RoomListing`. Khi không có document, con trỏ giữ giá trị trước đó. Cờ `hasMore` được đặt theo điều kiện số document bằng 20; đây là căn cứ cho phép thử tải tiếp, chưa khẳng định lượt kế tiếp chắc chắn còn dữ liệu. Lỗi truy vấn được chuyển tới callback lỗi, tách khỏi trường hợp truy vấn thành công nhưng danh sách rỗng.

`RoomViewModel` gọi `page()`, cập nhật con trỏ và ghép kết quả theo Room ID để tránh trùng phòng. Khi có từ khóa hoặc bộ lọc, ViewModel tự tải các trang còn lại trước khi kết luận không có kết quả; danh sách được phát qua `LiveData`; `MainActivity` quan sát kết quả để cập nhật danh sách và nút tải thêm.

Với danh sách booking, `BookingRepository.watchMine()` lọc `guestUid` bằng UID hiện tại qua hàm dùng chung `watchParticipant()`. Listener nhận thay đổi từ Firestore và chỉ chuyển snapshot đã được máy chủ xác nhận tới callback. Các document được ánh xạ thành `Booking`, sắp theo thời điểm tạo booking giảm dần rồi trả về ViewModel.

```
.whereEqualTo(field, uid)
.addSnapshotListener(
        MetadataChanges.INCLUDE,
        (rows, error) -> {
            if (error != null) {
                callback.onError(error);
                return;
            }
            // Chỉ phát dữ liệu đã xác nhận từ server, không phát cache hay ghi đang
            // chờ.
            if (rows != null
                    && !rows.getMetadata().isFromCache()
                    && !rows.getMetadata().hasPendingWrites())
```

Nguồn: `BookingRepository.java`, dòng 659 đến 671.

`BookingViewModel.watchMine()` đặt trạng thái tải, gọi Repository và cập nhật `bookings` bằng `UiState.success()` hoặc `UiState.error()`. `MyBookingsActivity` quan sát `LiveData` này để hiển thị dữ liệu và lọc trạng thái tại giao diện. Khi màn hình dừng, listener được gỡ; khi hiển thị lại, truy vấn được đăng ký lại để nhận các thay đổi tiếp theo.

4.3.3 Tạo booking bằng transaction

Khi người thuê gửi yêu cầu, `BookingRepository.create()` phải xác nhận phòng còn nhận đặt và khoảng thuê chưa bị chiếm. Booking ID được băm từ UID cùng `requestKey`; trường `requestFingerprint` biểu diễn nội dung yêu cầu. Nếu document theo ID này đã tồn tại và fingerprint trùng khớp, phương thức trả lại booking ID cũ. Nếu fingerprint khác, yêu cầu bị từ chối, tránh dùng lại cùng mã gửi cho một lựa chọn khác.

Trong callback `runTransaction()`, Repository đọc hồ sơ người thuê, phòng, bản chiếu Host, tòa nhà, cấu hình `appConfig/payments` và các tháng lịch liên quan. Booking mới phải thuộc Host của kênh demo với luồng `INSTANT_5M`. Tài khoản phải ACTIVE và có vai trò GUEST hoặc HOST; phòng phải APPROVED, ACTIVE và không thuộc chính người thuê. Host cùng tòa nhà phải còn hoạt động, đồng thời owner UID của Room phải khớp Building. Giá và thời gian thuê được tính lại từ dữ liệu phòng bằng `BookingPolicy.plan()`, sau đó lịch được kiểm tra trước mọi lệnh ghi.

```
        BookingPolicy.plan(room, request, System.currentTimeMillis());
DocumentSnapshot channel = tx.get(db.collection("appConfig").document("payments"));
BookingPolicy.require("INSTANT_5M".equals(channel.getString("flow"))
                && room.getOwnerUid().equals(channel.getString("demoHostUid")),
        "Phòng này chưa hỗ trợ thanh toán demo. Bạn có thể liên hệ Host.");
booking.paymentFlow = "INSTANT_5M";
booking.id = bookingId;
booking.guestUid = uid;
booking.guestName = guest.getString("name");
booking.guestPhone = guest.getString("phone");
booking.requestFingerprint = fingerprint;
List<AvailabilityMonth> months = months(tx, booking);
```

Nguồn: `BookingRepository.java`, dòng 206 đến 217.

Hàm `months()` đọc lịch tháng bằng chính transaction. Với thuê giờ, `reserve()` kiểm tra tập slot mới không giao với `slotsBusy`; với thuê cố định, tháng phải chưa có booking cố định hoặc booking giờ. Nếu có xung đột, hàm phát sinh lỗi trước bước ghi. Khi hợp lệ, các đối tượng lịch được cập nhật trong bộ nhớ rồi ghi cùng booking, revision của phòng và nhật ký.

```
for (int i = 0; i < months.size(); i++)
    tx.set(
            db.collection("availabilityMonths")
                    .document(booking.monthKeys.get(i)),
            months.get(i));
tx.set(ref, booking);
tx.update(
        db.collection("rooms").document(room.getId()),
        "activeBookingCount",
        room.activeBookingCount < 0 ? -1 : room.activeBookingCount + 1,
        "bookingRevision",
        FieldValue.increment(1),
```

Nguồn: `BookingRepository.java`, dòng 226 đến 237.

Firestore áp dụng toàn bộ các lệnh ghi trong transaction theo tính nguyên tử và có thể chạy lại callback khi document đã đọc bị thay đổi đồng thời (Firebase, n.d.-i). Vì các yêu cầu cùng phòng đọc và cập nhật lịch tháng chung, việc kiểm tra xung đột được thực hiện lại trên lịch mới khi transaction chạy lại. Callback thành công trả về booking ID sau khi giao dịch hoàn tất; lỗi kiểm tra, xung đột hoặc lỗi ghi được chuyển tới callback lỗi. Kết quả ghi thành công là booking PENDING và phần lịch tạm giữ; đây chưa phải xác nhận đã thanh toán. Màn hình QR được mở ngay để tiếp tục cùng booking.

Booking mới lưu `schemaVersion=2`, giá và chính sách tại thời điểm đặt trong `priceSnapshot`. Trường `paymentFlow=INSTANT_5M` xác định luồng giữ lịch tạm. Trước khi tạo yêu cầu payOS, booking chưa ghi nhận tiền; sau đó Payment theo dõi mã đơn, số tiền, QR và thời hạn. Backend đối chiếu kết quả nhà cung cấp, số tiền và thời điểm nhận tiền trước khi xác nhận `PAID/CONFIRMED`. Khi link đã kết thúc mà chưa trả tiền, backend chuyển booking sang `EXPIRED`, nhả đúng khóa lịch và giữ lịch sử. Tiền đến muộn, sai số tiền hoặc lịch không còn hợp lệ được đưa vào `REVIEW_REQUIRED`; chưa có API hoàn tiền tự động.

4.4 Phân quyền và bảo vệ dữ liệu

Firestore Rules kết hợp UID, vai trò, trạng thái ACTIVE và quyền sở hữu để kiểm soát truy cập. Guest và Host đều có quyền thuê, nhưng không được tự đặt phòng của mình. Bảng 4.4.1 tóm tắt các nhóm quyền chính; các thao tác ngoài nhánh được cho phép chịu quy tắc từ chối mặc định.

Bảng 4.4.1: Phân quyền theo nhóm dữ liệu

Nhóm	Quyền truy cập chính
Hồ sơ và đơn Host	Chủ hồ sơ hoặc Admin được đọc. Người dùng sửa nhóm trường hồ sơ cho phép; Guest nộp đơn, Admin xét duyệt. PublicHost được đọc bởi tài khoản hoạt động.
Alias và dữ liệu cá nhân	`usernames` cho phép get trước đăng nhập, list chỉ dành cho Admin. Favorite và mốc đã đọc thuộc quyền tài khoản sở hữu nhánh dữ liệu.

Nhóm	Quyền truy cập chính
Tòa nhà và phòng	Host quản lý tài sản của mình; Admin được cập nhật Room, không có nhánh ghi Building riêng. Phòng công khai cho tài khoản hoạt động khi APPROVED và ACTIVE.
Booking và lịch	Hai bên tham gia và Admin được đọc Booking. Chuyển trạng thái phụ thuộc vai trò, thời gian và lịch; Admin không có nhánh chuyển tùy ý. Ghi lịch phải khớp booking trong cùng thao tác.
Thanh toán và cấu hình	Người thuê, Host liên quan và Admin được đọc Payment; client không được tự ghi trạng thái tiền. PaymentOrder chỉ do backend truy cập; cấu hình kênh cho tài khoản hoạt động đọc, không chứa khóa payOS.
Chat	Chỉ thành viên đang hoạt động đọc hội thoại và message. Gửi tin cập nhật đồng thời preview, thứ tự và bộ đếm; mỗi bên chỉ tăng mốc đã đọc của mình. Admin không có quyền đọc riêng.
Quản trị và tham khảo	Chủ hồ sơ hoặc Admin đọc xác minh; Admin cập nhật kết quả. Báo cáo được đọc theo người gửi, Host liên quan hoặc Admin. Audit chỉ Admin đọc; RentalPlace chỉ cho tài khoản hoạt động đọc.

Quyền ghi còn phụ thuộc nội dung và tính nhất quán giữa document. Rules dùng danh sách trường cho phép và `diff().affectedKeys()` để giới hạn thay đổi; `getAfter()` kiểm tra trạng thái sau ghi của các document liên quan (Firebase, n.d.-c, n.d.-j). Ví dụ, `heldMonth()` đối chiếu booking với lịch tháng trước khi chấp nhận thao tác giữ chỗ.

```
function heldMonth(b,i) {
  let id=monthId(b,i);
  let m=getAfter(/databases/$(database)/documents/availabilityMonths/$(id)).data;
  return b.monthKeys[i]==id && m.roomId==b.roomId
    && (b.activeMode=='FIXED' ? m.fixedBookingId==b.id && m.slotsBusy.size()==0
        : m.fixedBookingId=='' && m.hourlyBookings[b.id]==b.slotKeys &&
        ↪  m.slotsBusy.hasAll(b.slotKeys));
}
```

Nguồn: `firestore.rules`, dòng 24 đến 30.

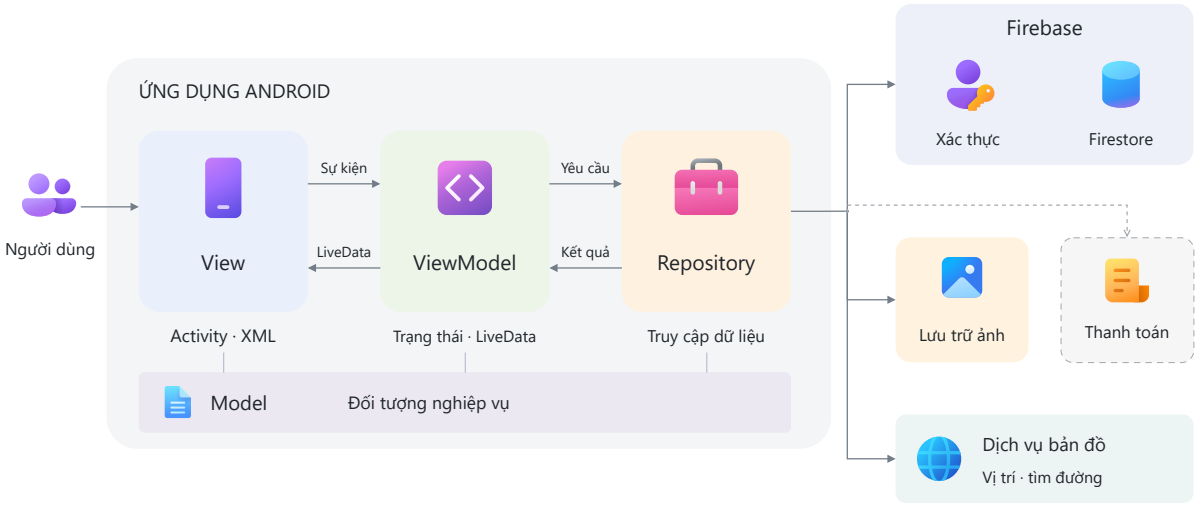
Backend dùng Firebase Admin SDK nên phải tự xác thực Firebase ID token, kiểm tra người tham gia và đối chiếu kết quả payOS; không dựa vào Rules để xác thực lệnh ghi của server (Firebase, n.d.-j). Khóa dịch vụ được giữ ở môi trường backend. Ảnh và minh chứng DEMO hiện dùng URL Cloudinary; Rules kiểm tra đường dẫn theo UID và đối tượng liên quan. Ảnh giấy tờ DEMO công khai theo URL không được xem là cơ chế bảo vệ giấy tờ cá nhân thật.

5. XÂY DỰNG ỨNG DỤNG VÀ THIẾT KẾ KIẾN TRÚC

5.1 Tổng quan kiến trúc Java + XML, MVVM + Repository + Model

BroomSharing lựa chọn định hướng tổ chức ứng dụng thành các thành phần giao diện, trạng thái màn hình và truy cập dữ liệu. Định hướng này phù hợp với các khuyến nghị về phân tầng giao diện, dữ liệu và quản lý trạng thái của Android Developers (n.d.-b, n.d.-d, n.d.-e).

View gồm các tệp XML, Activity/Fragment và Adapter, chịu trách nhiệm hiển thị danh sách, biểu mẫu, thông báo và tiếp nhận sự kiện người dùng. **ViewModel** nhận hành động từ View, xử lý trạng thái phục vụ giao diện, yêu cầu dữ liệu và cung cấp dữ liệu có thể quan sát. **Repository** cung cấp các thao tác làm việc với không gian, người dùng và booking, che giấu chi tiết gọi Firestore. **Model** gồm các lớp Java biểu diễn dữ liệu nghiệp vụ của ứng dụng.



Hình 5.1.1: Kiến trúc tổng quan của ứng dụng BroomSharing

Mũi tên từ View đến ViewModel biểu diễn lời gọi phương thức xử lý tương tác; chiều ngược lại biểu diễn dữ liệu trạng thái để View quan sát. ViewModel giao tiếp với Repository, không cần phụ thuộc vào API Firestore. Model được các thành phần sử dụng để biểu diễn dữ liệu nghiệp vụ. Nhánh thanh toán dùng nét đứt để thể hiện phần tích hợp dự kiến, chưa xác định nhà cung cấp.

5.2 Cấu trúc thư mục đề xuất

Bảng 5.2.1: Tổ chức gói mã nguồn Java + XML theo trách nhiệm

Nhóm thư mục	Nhiệm vụ
`ui/`	Activity, Fragment, Adapter, ViewHolder và thành phần điều hướng.
`viewmodel/`	Ví dụ `SpaceViewModel`, `BookingViewModel`; cung cấp LiveData hoặc trạng thái tương ứng.
`repository/`	Ví dụ `SpaceRepository`, `BookingRepository`; đọc/ghi dữ liệu qua Firebase SDK.
`model/`	Các POJO `User`, `Space`, `Booking`, `Review` nếu cần.
`res/layout/`	XML cho Activity, Fragment, item danh sách và hộp thoại.
`res/drawable/`, `res/values/`	Ảnh, icon, màu, chuỗi và theme của ứng dụng.

5.3 Kết nối với Firebase

Ứng dụng Android cần được liên kết với dự án Firebase, cấu hình thư viện theo hướng dẫn chính thức và kiểm tra quyền đọc/ghi. Repository là nơi phù hợp để gọi `FirebaseFirestore` qua phương thức `getInstance()`, thực hiện truy vấn và ánh xạ tài liệu sang đối tượng Java. Hoạt động bất đồng bộ phải trả kết quả bằng callback/LiveData hoặc cơ chế tương đương; không thực hiện chờ mạng trên luồng giao diện (Firebase, n.d.-a, n.d.-e).

Đối với ảnh không gian, nên phân biệt tệp ảnh với bản ghi dữ liệu: tệp được quản lý bởi dịch vụ lưu trữ phù hợp; URL hoặc đường dẫn được tham chiếu trong `spaces`. Nếu phiên bản đồ án chỉ dùng ảnh tĩnh trong `drawable`, báo cáo phải mô tả đúng lựa chọn đó thay vì khẳng định có tải ảnh lên Storage.

5.4 Minh họa luồng nghiệp vụ "xem danh sách không gian"

1. Activity/Fragment tạo hoặc lấy ViewModel gắn với vòng đời giao diện.
2. View đăng ký quan sát danh sách không gian qua LiveData.
3. ViewModel gọi phương thức lấy danh sách ở SpaceRepository.
4. Repository truy vấn Firestore, ánh xạ document thành các đối tượng Space.
5. Khi có dữ liệu hoặc lỗi, ViewModel cập nhật trạng thái; View hiển thị thông tin qua RecyclerView hoặc trạng thái rỗng/lỗi tương ứng.

Luồng này là **phương án tổ chức mã nguồn kiến trúc**; chưa phải bằng chứng rằng toàn bộ Activity hiện tại đã được refactor theo MVVM.

5.5 Tích hợp bản đồ và ảnh trong kiến trúc

Phần bản đồ và quản lý ảnh được bố trí như các dịch vụ tích hợp, không thay đổi nguyên tắc phân lớp chính của MVVM. Tab **Map** sử dụng `MapView` của MapLibre trong XML và Activity/Fragment Java để hiển thị bản đồ. Repository truy xuất danh sách không gian có tọa độ từ Firestore; lớp giao diện nhận dữ liệu đã được ViewModel cung cấp và đặt marker. Khi chọn tìm đường, thành phần tích hợp openrouteservice nhận điểm đi/đến, đọc GeoJSON và chuyển hình học tuyến để hiển thị trên MapLibre. MapTiler cung cấp style/tile và có thể hỗ trợ chuyển đổi địa chỉ thành tọa độ (GIScience, n.d.; MapLibre, n.d.; MapTiler, n.d.).

Ảnh không gian được chọn ở giao diện, tải lên Cloudinary theo cấu hình cho phép; sau khi thành công, ứng dụng cập nhật URL vào tài liệu `spaces` trên Firestore. Chỉ nên đưa vào báo cáo cuối kỳ các bước đã có mã nguồn và kết quả chạy. Việc gọi API mạng phải bất đồng bộ; không lưu khóa bí mật dịch vụ ở dạng hard-code trong ứng dụng (Cloudinary, n.d.).

Bảng 5.5.1: Vai trò các công nghệ và luồng phụ trợ trong kiến trúc

Thành phần	**Trách nhiệm trong thiết kế**
MapLibre Native	Thành phần Android View để hiển thị bản đồ, marker và đường đi.
MapTiler	Cung cấp dữ liệu bản đồ và geocoding theo API key.
openrouteservice	Trả dữ liệu tuyến đường giữa các tọa độ, ví dụ GeoJSON.
Android Location	Lấy tọa độ của thiết bị khi đã có quyền phù hợp.
Cloudinary	Upload và lưu trữ tệp ảnh, trả URL để ghi vào Firestore.
Cloud Firestore	Lưu các bản ghi nghiệp vụ, thuộc tính tọa độ và URL ảnh.

5.6 Các màn hình và mức độ hoàn thiện

Màn hình khám phá, chi tiết, bản đồ, đặt chỗ, đăng không gian, lịch sử booking, tin nhắn, hồ sơ và quản trị đã được thảo luận ở giai đoạn thiết kế UI. Để ghi nhận là *đã xây dựng*, từng màn hình cần có tệp layout/Activity thật, ảnh chụp và một mục kiểm thử. Giai đoạn thiết kế có thể chấp nhận việc một số màn hình chỉ có bố cục XML hoặc dữ liệu mẫu; giai đoạn báo cáo thực thi phải mô tả khác biệt này rõ ràng.

6. QUY TRÌNH THỰC THI VÀ KIỂM THỬ

6.1 Quy trình đăng ký và đăng nhập

Người dùng nhập thông tin cơ bản, hệ thống kiểm tra dữ liệu hợp lệ và xác lập phiên đăng nhập. Sau khi đăng nhập, giao diện có thể điều chỉnh theo quyền User hoặc Owner. Nếu tích hợp Firebase Authentication, danh tính đăng nhập cần liên hệ nhất quán với document hồ sơ trong `users`; tài khoản không được coi là hợp lệ chỉ dựa vào dữ liệu do client tự khai báo (Firebase, n.d.-f).

6.2 Quy trình tìm kiếm và đặt thuê không gian

1. **Khám phá:** người dùng mở danh sách, tra cứu từ khóa hoặc lọc địa điểm, giá, kỳ thuê.
2. **Đối chiếu:** xem mô tả, ảnh, giá và điều kiện sử dụng của không gian được chọn.
3. **Chọn thời gian:** nhập ngày bắt đầu/kết thúc theo quy ước đặt phòng hoặc kỳ thuê.
4. **Kiểm tra:** đánh giá tính hợp lệ, điều kiện sẵn có và giá dự kiến.
5. **Gửi yêu cầu:** tạo document booking với trạng thái ban đầu phù hợp.
6. **Theo dõi:** hiển thị trạng thái cập nhật trong lịch sử đặt chỗ.

6.3 Quy trình đăng và quản lý không gian

Chủ không gian tạo bản ghi mới bằng cách nhập tên, địa chỉ, ảnh, tiện ích, thời gian sẵn có và giá theo đơn vị quy định. Trước khi công bố, thông tin bắt buộc cần hợp lệ. Khi nhận booking, chủ không gian xem chi tiết yêu cầu, cập nhật trạng thái theo quyền được phép và theo dõi các mốc thời gian đã được xác nhận. Nếu áp dụng duyệt tin bởi Admin, trạng thái "chờ duyệt" sẽ được xét riêng thay vì công bố trực tiếp.

6.4 Quy trình xác nhận và QR check-in

Sau khi booking được chấp thuận, hệ thống có thể cung cấp một mã tham chiếu hoặc mã QR cho bước nhận không gian. Phương án QR cần gắn với booking cụ thể, có hạn sử dụng và chỉ chấp nhận xác nhận một lần theo đúng chính sách. Chức năng này được giữ ở mức **định hướng nghiệp vụ**, do chưa có căn cứ về việc quét, sinh mã và xác nhận tại thiết bị đã được cài đặt đầy đủ.

6.5 Thiết kế các ca kiểm thử

Các bảng dưới đây là bộ ca kiểm thử cần thực hiện trên emulator/thiết bị và tài khoản thử nghiệm. Cột kết quả được để trống có chủ ý: chỉ ghi nhận sau khi thu thập ảnh hoặc log thực thi.

Bảng 6.5.1: Ma trận kiểm thử chức năng

Mã	Tình huống	Kết quả mong đợi	Kết quả thực tế
TC-01	Vào màn hình khám phá khi có dữ liệu.	Danh sách hiển thị đầy đủ thông tin chính.	Chưa kiểm thử
TC-02	Bộ lọc không khớp tin nào.	Hiện thông báo rỗng, không crash.	Chưa kiểm thử
TC-03	Mở màn hình chi tiết từ một item.	Hiển thị đúng mã không gian được chọn.	Chưa kiểm thử
TC-04	Chọn ngày kết thúc trước ngày bắt đầu.	Chặn yêu cầu, giải thích lỗi.	Chưa kiểm thử
TC-05	Đặt trùng khung thời gian đã xác nhận.	Không phát sinh booking xác nhận trùng.	Chưa kiểm thử
TC-06	Owner sửa tin của người khác.	Hệ thống từ chối ghi dữ liệu.	Chưa kiểm thử
TC-07	Mất kết nối khi gửi yêu cầu.	Hiển thị trạng thái lỗi/thử lại phù hợp.	Chưa kiểm thử
TC-08	Đăng tin thiếu tên hoặc giá.	Báo trường bắt buộc, không ghi sai dữ liệu.	Chưa kiểm thử
TC-09	Xoay màn hình khi đang xem danh sách.	Trạng thái UI không sai lệch nghiêm trọng.	Chưa kiểm thử
TC-10	QR đã dùng hoặc hết hạn (nếu tích hợp).	Không chấp nhận tái sử dụng.	Chưa kiểm thử
TC-11	Bật bản đồ và cho phép truy cập vị trí.	Đọc tọa độ hợp lệ, hiển thị marker không gian.	Chưa kiểm thử
TC-12	Yêu cầu chỉ đường đến không gian.	Dữ liệu tuyến trả về được xử lý hoặc hiển thị lỗi mạng phù hợp.	Chưa kiểm thử
TC-13	Thêm ảnh từ thiết bị lên Cloudinary.	Chỉ ghi URL vào Firestore sau khi upload thành công.	Chưa kiểm thử

6.6 Đánh giá giới hạn và chứng cứ thực thi

Một báo cáo hoàn chỉnh cần có hình chụp giao diện từ thiết bị/emulator, một ví dụ document Firestore đã lưu và minh chứng kiểm tra ít nhất một quy trình đầu-cuối. Ở bản dựng này, thông tin đó chưa có trong tài liệu đầu vào, vì vậy chưa thể kết luận về tỷ lệ kiểm thử đạt, độ ổn định hay thời gian đáp ứng của hệ thống. Không gán các chỉ số giả định để thay thế cho kết quả đo thật.

7. KẾT LUẬN VÀ HƯỚNG MỞ RỘNG

7.1 Kết luận của giai đoạn thiết kế

BroomSharing đề xuất một mô hình ứng dụng Android để khai thác và chia sẻ không gian còn trống theo ngày hoặc tháng. Báo cáo xác định được nhóm tác nhân User, Owner và Admin (mở rộng), các thao tác chính gồm tìm kiếm, đọc chi tiết, đăng không gian, gửi yêu cầu thuê và quản lý booking. Lược đồ dữ liệu dự kiến tập trung vào các collection `users`, `spaces` và `bookings`, cho phép mô tả rõ các liên kết logic cốt lõi.

Về kiến trúc, cách tổ chức Java + XML theo MVVM + Repository + Model giúp tách công việc hiển thị giao diện khỏi việc truy cập Firebase (Android Developers, n.d.-b, n.d.-c). Tuy nhiên, đây là kết quả phân tích và thiết kế, không thay cho việc kiểm tra implementation, kiểm thử và khả năng hoạt động thực tế của ứng dụng ở thời điểm nộp bài.

7.2 Những hạn chế hiện tại

Bản báo cáo được biên soạn từ tài liệu mẫu môn học và thông tin nghiệp vụ đã trao đổi, không được cung cấp trực tiếp mã nguồn BroomSharing hoặc bản xuất Firestore để đối chiếu. Vì vậy tên lớp, bộ sưu tập dữ liệu, trạng thái booking và sự hiện diện của các màn hình là phương án đề xuất. Không có căn cứ xác nhận thanh toán trong ứng dụng, đồng bộ lịch theo thời gian thực, chat hoàn chỉnh, QR check-in, quy trình duyệt tin hoặc kiểm thử thiết bị đã hoàn thành.

7.3 Hướng phát triển

- Hoàn thiện luồng bản đồ MapLibre, marker từ Firestore, MapTiler geocoding và openrouteservice directions; kiểm chứng giới hạn API key và quota trên môi trường thử nghiệm.
- Hoàn thiện upload ảnh Cloudinary có kiểm soát, thử nghiệm các tình huống lỗi mạng và giới hạn dung lượng.
- Hoàn thiện bộ lọc theo vị trí, giá, kỳ thuê và tiện ích, đồng thời tối ưu dữ liệu tìm kiếm.
- Xây dựng cơ chế kiểm tra chồng lấn booking đáng tin cậy tại tầng dữ liệu hoặc backend.
- Cải thiện chất lượng tin đăng bằng quy trình duyệt nội dung và báo cáo thông tin không chính xác.
- Bổ sung thông báo trạng thái booking, đánh giá sau sử dụng và lịch sử giao dịch.
- Nghiên cứu QR check-in/check-out có điều kiện và cơ chế ngăn sử dụng lại mã.
- Thực hiện kiểm thử chức năng, bảo mật và hiệu năng trước khi phát hành thử nghiệm.

TÀI LIỆU THAM KHẢO

Android Developers. (n.d.-a). *Create dynamic lists with RecyclerView*. Google. Retrieved October 20, 2026, from https://developer.android.com/develop/ui/views/layout/recyclerview

Android Developers. (n.d.-b). *Data layer*. Google. Retrieved October 20, 2026, from https://developer.android.com/topic/architecture/data-layer

Android Developers. (n.d.-c). *Guide to app architecture*. Google. Retrieved October 20, 2026, from https://developer.android.com/topic/architecture

Android Developers. (n.d.-d). *Recommendations for Android architecture (Views)*. Google. Retrieved October 20, 2026, from https://developer.android.com/topic/architecture/views/recommendations-views

Android Developers. (n.d.-e). *ViewModel overview*. Google. Retrieved October 20, 2026, from https://developer.android.com/topic/libraries/architecture/viewmodel.html

Cloudinary. (n.d.). *Android image and video upload*. Cloudinary. Retrieved October 20, 2026, from https://cloudinary.com/documentation/android_image_and_video_upload

Firebase. (n.d.-a). *Add Firebase to your Android project*. Google. Retrieved October 20, 2026, from https://firebase.google.com/docs/android/setup

Firebase. (n.d.-b). *Cloud Firestore data model*. Google. Retrieved October 23, 2026, from https://firebase.google.com/docs/firestore/data-model

Firebase. (n.d.-c). *Control access to specific fields*. Google. Retrieved October 23, 2026, from https://firebase.google.com/docs/firestore/security/rules-fields

Firebase. (n.d.-d). *Get realtime updates with Cloud Firestore*. Google. Retrieved October 23, 2026, from https://firebase.google.com/docs/firestore/query-data/listen

Firebase. (n.d.-e). *Get started with Cloud Firestore*. Google. Retrieved October 20, 2026, from https://firebase.google.com/docs/firestore/quickstart

Firebase. (n.d.-f). *Get started with Cloud Firestore Security Rules*. Google. Retrieved October 20, 2026, from https://firebase.google.com/docs/firestore/security/get-started

Firebase. (n.d.-g). *Paginate data with query cursors*. Google. Retrieved October 24, 2026, from https://firebase.google.com/docs/firestore/query-data/query-cursors

Firebase. (n.d.-h). *Perform simple and compound queries in Cloud Firestore*. Google. Retrieved October 23, 2026, from https://firebase.google.com/docs/firestore/query-data/queries

Firebase. (n.d.-i). *Transactions and batched writes*. Google. Retrieved October 20, 2026, from https://firebase.google.com/docs/firestore/manage-data/transactions

Firebase. (n.d.-j). *Writing conditions for Cloud Firestore Security Rules*. Google. Retrieved October 23, 2026, from https://firebase.google.com/docs/firestore/security/rules-conditions

GIScience. (n.d.). *Openrouteservice: Directions requests and return types*. Heidelberg University. Retrieved October 20, 2026, from https://github.com/GIScience/openrouteservice/blob/main/docs/api-reference/endpoints/directions/requests-and-return-types.md

MapLibre. (n.d.). *MapLibre Native Android API*. MapLibre. Retrieved October 20, 2026, from https://maplibre.org/maplibre-native/android/api/

MapTiler. (n.d.). *Search and geocoding API*. MapTiler. Retrieved October 20, 2026, from https://docs.maptiler.com/cloud/api/geocoding/

Nhóm phát triển BroomSharing. (n.d.). BroomSharing: Mã nguồn Android Java và cấu hình Firebase [app/src/main/java; firestore.rules; storage.rules; firestore.indexes.json].

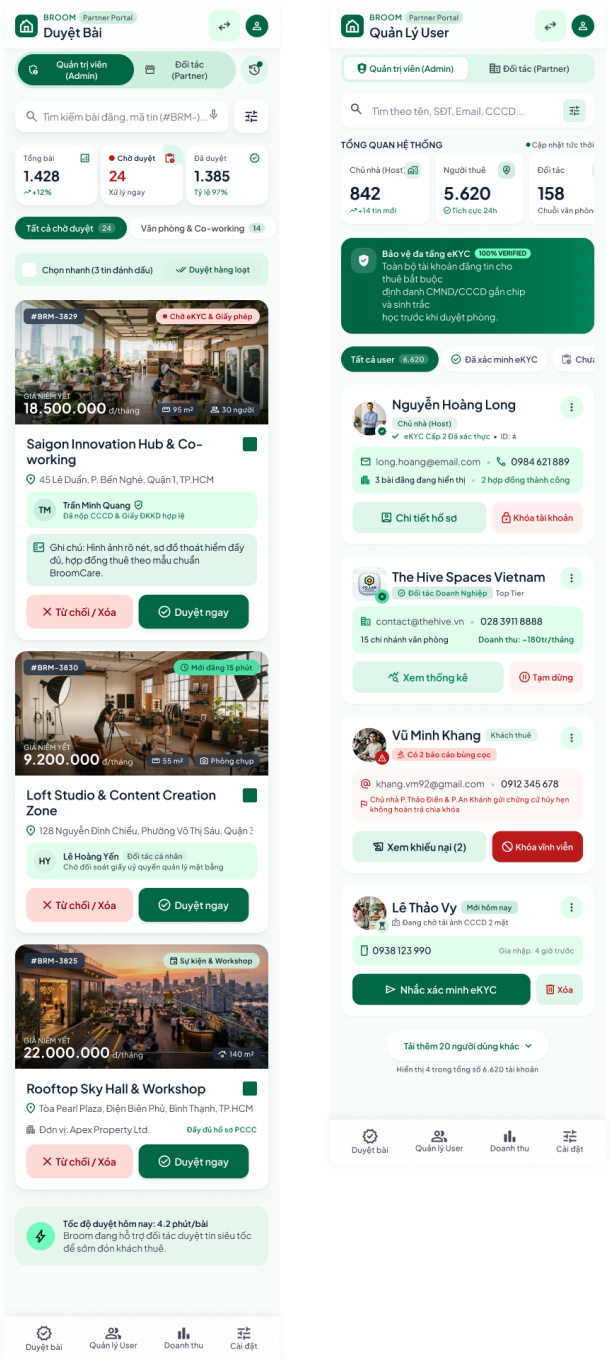
PHỤ LỤC A. HÌNH ẢNH

Hình 2.2.1 Quy trình cấp quyền Host và công bố không gian. 6
Hình 2.2.2 Quy trình tạo, giữ lịch và xử lý yêu cầu đặt chỗ. 7
Hình 2.2.3 Quy trình sử dụng phòng, khép lượt thuê và xác minh ngoại lệ. . . . 8
Hình 2.2.4 Luồng tạo QR và cập nhật kết quả thanh toán payOS. 9
Hình 2.3.1 Phân rã chức năng tổng quát của BroomSharing. 10
Hình 2.4.1 Use case tổng quát với các tác nhân người thuê, Host, Admin và dịch vụ thanh toán. 11
Hình 2.4.2 Use case chi tiết của người thuê. 12
Hình 2.4.3 Use case chi tiết của Host. 13
Hình 2.4.4 Use case chi tiết của Admin. 14
Hình 3.2.1 Thiết kế biểu mẫu đăng nhập và đăng ký tài khoản. 18
Hình 3.2.2 Thiết kế Trang chủ. 19
Hình 3.2.3 Thiết kế thông tin chi tiết không gian. 19
Hình 3.2.4 Prototype xác nhận thông tin và chi phí đặt phòng. 20
Hình 3.3.1 Prototype ba bước nhập thông tin và đăng không gian. 22
Hình 4.2.1 Cây lưu trữ Firestore BroomSharing. Thẻ màu xám là contract thông báo cũ trong Rules. 25
Hình 4.2.2 Cấu trúc nhóm tài khoản và các subcollection thuộc hồ sơ người dùng. 26
Hình 4.2.3 Cấu trúc tài sản, booking, lịch tháng, thanh toán và đánh giá. Các collection nằm ngang cấp trong database. 27
Hình 4.2.4 Cấu trúc hội thoại và subcollection tin nhắn của từng hội thoại. . . . 28
Hình 4.2.5 Nhóm quản trị, dữ liệu tham khảo và cấu hình backend. [D] là trường từ dữ liệu mẫu; Rules-only là contract chưa có luồng ghi hiện tại. 29
Hình 5.1.1 Kiến trúc tổng quan của ứng dụng BroomSharing 36
Hình A.1 Phương án thiết kế kiểm duyệt và quản lý trong phân hệ Admin. . . 47
Hình G.1 Phân rã chức năng: tài khoản và phiên. 55
Hình G.2 Phân rã chức năng: khám phá không gian. 56
Hình G.3 Phân rã chức năng: tạo yêu cầu đặt chỗ. 57
Hình G.4 Phân rã chức năng: lịch đặt và hủy. 58
Hình G.5 Phân rã chức năng: hồ sơ host và xác minh. 59
Hình G.6 Phân rã chức năng: quản lý không gian. 60
Hình G.7 Phân rã chức năng: theo dõi và sử dụng. 61
Hình G.8 Phân rã chức năng: chat và liên hệ. 62
Hình G.9 Phân rã chức năng: đánh giá sau sử dụng. 63

Hình G.10 Phân rã chức năng: báo cáo và phản hồi. 64
Hình G.11 Phân rã chức năng: quản trị và sự cố. 65
Hình G.12 Phân rã chức năng: theo dõi và nhật ký. 66
Hình G.13 Phân rã chức năng: hồ sơ và thông báo. 67
Hình G.14 Phân rã chức năng: thanh toán demo. 68

A.1. Thiết kế giao diện Admin

Hình A.1 minh họa hai nhóm nội dung kiểm duyệt phòng và quản lý tài khoản. Thẻ thông tin đặt thao tác chấp thuận, từ chối hoặc quản lý trạng thái cạnh đối tượng cần xử lý. Phần tổng quan và bộ lọc hỗ trợ xác định nhóm hồ sơ trước khi mở chi tiết; số liệu trong hình là dữ liệu minh họa thiết kế.



Hình A.1: Phương án thiết kế kiểm duyệt và quản lý trong phân hệ Admin.

PHỤ LỤC B. TRUY VẤN SQL VÀ TRUY VẤN FIRESTORE

Phụ lục minh họa các thao tác truy vấn dữ liệu trong BroomSharing. Vì kiến trúc sử dụng Cloud Firestore (NoSQL), các câu SQL chỉ biểu diễn logic tương đương, không phải mã có thể thực thi trực tiếp trên Firestore.

B.1. Liệt kê không gian đang được công bố

```sql
-- Pseudo-SQL: minh hoa truy van tuong duong
SELECT space_id, title, address, price, status
FROM spaces
WHERE status = 'available'
ORDER BY created_at DESC;
```

Lệnh Java/Firestore tương ứng (minh họa):

```java
FirebaseFirestore db = FirebaseFirestore.getInstance();
db.collection("spaces")
  .whereEqualTo("status", "available")
  .orderBy("createdAt", Query.Direction.DESCENDING)
  .get()
  .addOnSuccessListener(result -> {
      // Anh xa document sang Model Space
  });
```

Lưu ý: tổ hợp lọc và sắp xếp có thể đòi hỏi tạo composite index trên Firestore theo hướng dẫn của Firebase.

B.2. Tìm kiếm không gian theo mức giá

```sql
-- Pseudo-SQL: minh hoa logic loc theo muc gia
SELECT space_id, title, price
FROM spaces
WHERE status = 'available'
  AND price BETWEEN :min_price AND :max_price
ORDER BY price ASC;
```

Hướng truy vấn Firestore: dùng `whereGreaterThanOrEqualTo`, `whereLessThanOrEqualTo` và `orderBy` trên trường giá. Trước khi triển khai phải thống nhất đơn vị tiền tệ và kỳ thuê để dữ liệu có thể so sánh được.

B.3. Danh sách booking của một người dùng

```sql
-- Pseudo-SQL: minh hoa logic truy van
SELECT booking_id, space_id, start_time, end_time, status
FROM bookings
WHERE renter_id = :current_user_id
ORDER BY created_at DESC;
```

Hướng truy vấn Firestore: lọc theo `renterId` của người dùng hiện tại, sau đó sắp xếp theo `createdAt`; quy tắc Security Rules phải kiểm tra quyền đọc của tài khoản. Thao tác đặt trùng thời gian cần được xử lý nhất quán và không chỉ dựa vào kiểm tra ở giao diện (Firebase, n.d.-f, n.d.-i).

B.4. Liệt kê không gian do một chủ sở hữu quản lý

```sql
-- Pseudo-SQL: danh sach tin dang cua Owner
SELECT space_id, title, price, status, created_at
FROM spaces
WHERE owner_id = :current_owner_id
ORDER BY created_at DESC;
```

Hướng truy vấn Firestore: sử dụng điều kiện lọc `ownerId` theo định danh đăng nhập và sắp xếp `createdAt`. Quyền sửa tin cần được kiểm tra bằng Security Rules thay vì chỉ kiểm soát nút trên giao diện (Firebase, n.d.-f).

B.5. Cập nhật trạng thái yêu cầu đặt thuê

```sql
-- Pseudo-SQL: Owner xu ly yeu cau booking
UPDATE bookings
SET status = 'approved', updated_at = :now
WHERE booking_id = :booking_id
  AND status = 'pending'
  AND owner_id = :current_owner_id;
```

Hướng xử lý Firestore: trước khi chuyển trạng thái, xác nhận quyền của chủ không gian và trạng thái hiện tại; nếu có dữ liệu phụ thuộc hoặc điều kiện cạnh tranh, xét cơ chế transaction và kiểm tra tính nhất quán (Firebase, n.d.-f, n.d.-i). Câu truy vấn trên chỉ dùng để giải thích điều kiện nghiệp vụ.

PHỤ LỤC C. DANH MỤC BẢNG, BIỂU

Danh mục bảng biểu trong phần thân báo cáo được sinh tự động; số trang và số thứ tự sẽ cập nhật theo kết quả biên dịch cuối cùng.

Bảng 2.1.1 Tác nhân và phạm vi chức năng 4
Bảng 4.1.1 Các mẫu document và thuộc tính tiêu biểu 23
Bảng 4.2.1 Các quan hệ logic chính 30
Bảng 4.3.1 Các thao tác dữ liệu tiêu biểu trong ứng dụng 31
Bảng 4.4.1 Phân quyền theo nhóm dữ liệu 34
Bảng 5.2.1 Tổ chức gói mã nguồn Java + XML theo trách nhiệm 37
Bảng 5.5.1 Vai trò các công nghệ và luồng phụ trợ trong kiến trúc 38
Bảng 6.5.1 Ma trận kiểm thử chức năng 40
Bảng PL.1 Bảng nhiệm vụ nhóm cần xác nhận 52

PHỤ LỤC D. ĐƯỜNG DẪN TÀI NGUYÊN ĐỒ ÁN

Các tài nguyên trực tuyến hỗ trợ theo dõi tiến độ, đối chiếu mã nguồn và kiểm tra minh chứng thực hiện của nhóm.

Bảng phân công nhiệm vụ

Bảng phân công được nhóm lưu trực tuyến để giảng viên theo dõi nhiệm vụ, tiến độ và kết quả của từng thành viên.

XEM BẢNG PHÂN CÔNG CÔNG VIỆC

Link đồ án – GitHub

Mã nguồn BroomSharing được quản lý bằng GitHub nhằm phục vụ việc xem xét các thành phần dự án và lịch sử phát triển.

XEM SOURCE CODE BROOMSHARING

Đường dẫn học phần

Các tài liệu và yêu cầu được giảng viên công bố là cơ sở để đối chiếu cấu trúc và tiêu chí đánh giá đồ án.

XEM THÔNG TIN HỌC PHẦN

GitHub Projects và thư mục minh chứng

Bảng công việc và tập ảnh ghi nhận tiến độ được quản lý độc lập với báo cáo để thuận tiện cập nhật.

XEM NHẬT KÝ PHÂN CÔNG

XEM THƯ MỤC ẢNH MINH CHỨNG

PHỤ LỤC E. BẢNG PHÂN CÔNG VÀ MINH CHỨNG NHÓM

Bảng phân công nêu trách nhiệm và sản phẩm bàn giao của từng thành viên; minh chứng cần đối chiếu được với GitHub Projects, Google Sheets hoặc lịch sử commit.

Bảng PL.1: Bảng nhiệm vụ nhóm cần xác nhận

Thành viên	**Nhiệm vụ riêng**	**Commit / Issues / Sheet**
Lữ Võ Hoàng Phúc	Bổ sung phần việc thực tế theo log triển khai BroomSharing.	Chưa cung cấp
Lâm Hoàng	Ghi đúng hạng mục đã nhận và kết quả bàn giao.	Chưa cung cấp
Từ Trương Thanh Trúc	Ghi đúng hạng mục đã nhận và kết quả bàn giao.	Chưa cung cấp

Minh chứng nên kèm: ảnh phân công GitHub Projects hoặc Google Sheets; ảnh lịch sử commit thể hiện tên người thực hiện, thời gian và tệp thay đổi; liên kết commit tương ứng. Không suy đoán kết quả của từng thành viên khi chưa có dữ liệu.

PHỤ LỤC F. TƯ LIỆU VÀ KIỂM TRA BỔ SUNG

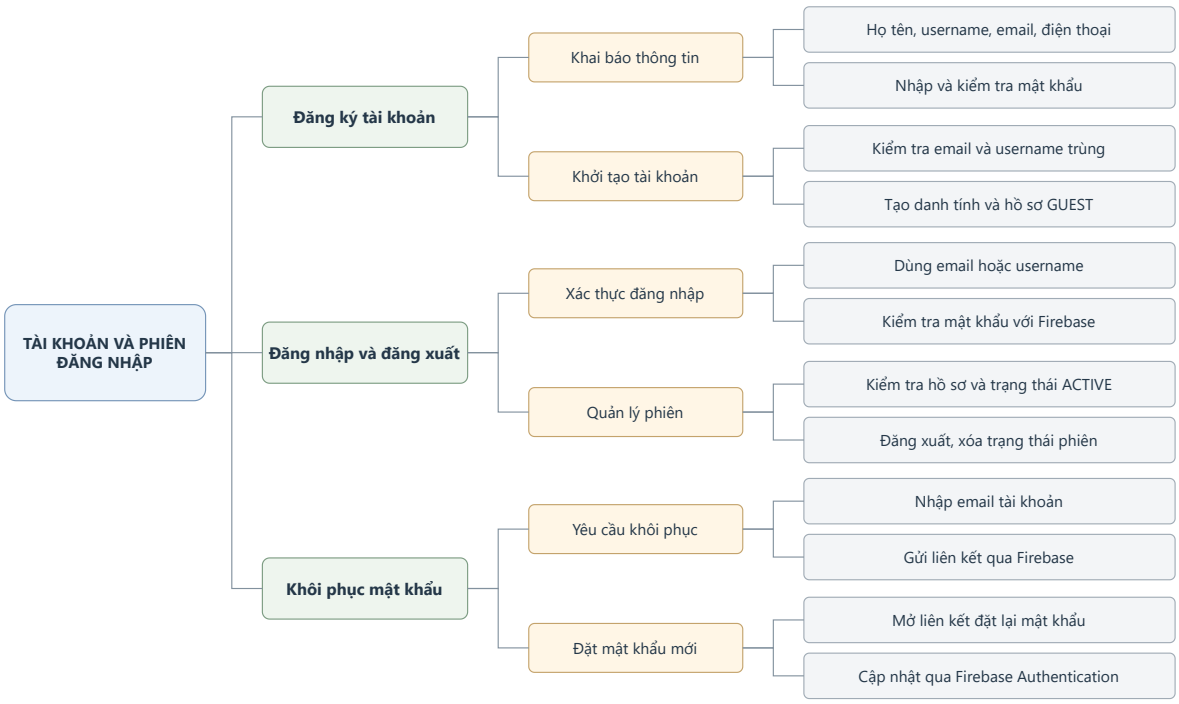
Các tài liệu vượt khỏi dung lượng báo cáo chính hoặc minh chứng phát sinh trong quá trình triển khai được đặt tại đây để không làm gián đoạn dòng trình bày của bảy chương.

1. Nhật ký thử nghiệm trên emulator và thiết bị thực: phiên bản Android, loại thiết bị, thao tác thực hiện, kết quả, ảnh/log.
2. Trích xuất cấu trúc collection/document Firestore và Security Rules đã kiểm tra; không công khai khóa bí mật.
3. Ảnh chụp bổ sung về màn hình tìm kiếm, hồ sơ, chủ không gian, xử lý booking, bản đồ hoặc QR (chỉ khi đã thực hiện).
4. Các đoạn truy vấn hoặc mã Java/Repository quá dài: chuyển sang mã nguồn GitHub và liên kết đến dòng code tương ứng.
5. Kịch bản kiểm thử bổ sung, lỗi đã phát hiện và cách khắc phục; ghi tách biệt với chức năng chỉ mới được đề xuất.

PHỤ LỤC G. PHÂN RÃ CHỨC NĂNG CHI TIẾT

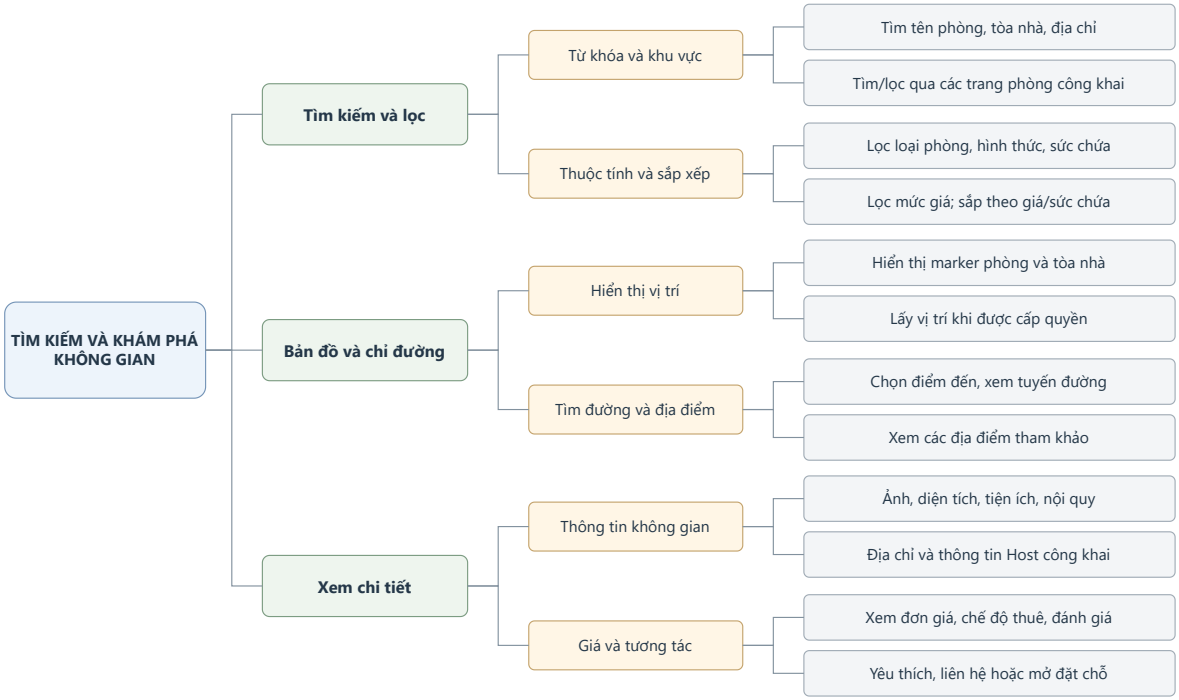
Các nhóm chức năng dưới đây triển khai chi tiết từ sơ đồ tổng quát ở Chương 2. Mỗi nhóm gắn với một phạm vi nghiệp vụ; các bước dùng chung được giữ trong cùng nhóm để thể hiện trách nhiệm xử lý.

Nhóm tài khoản quản lý đăng nhập, đăng ký, khôi phục mật khẩu và phiên truy cập theo vai trò.



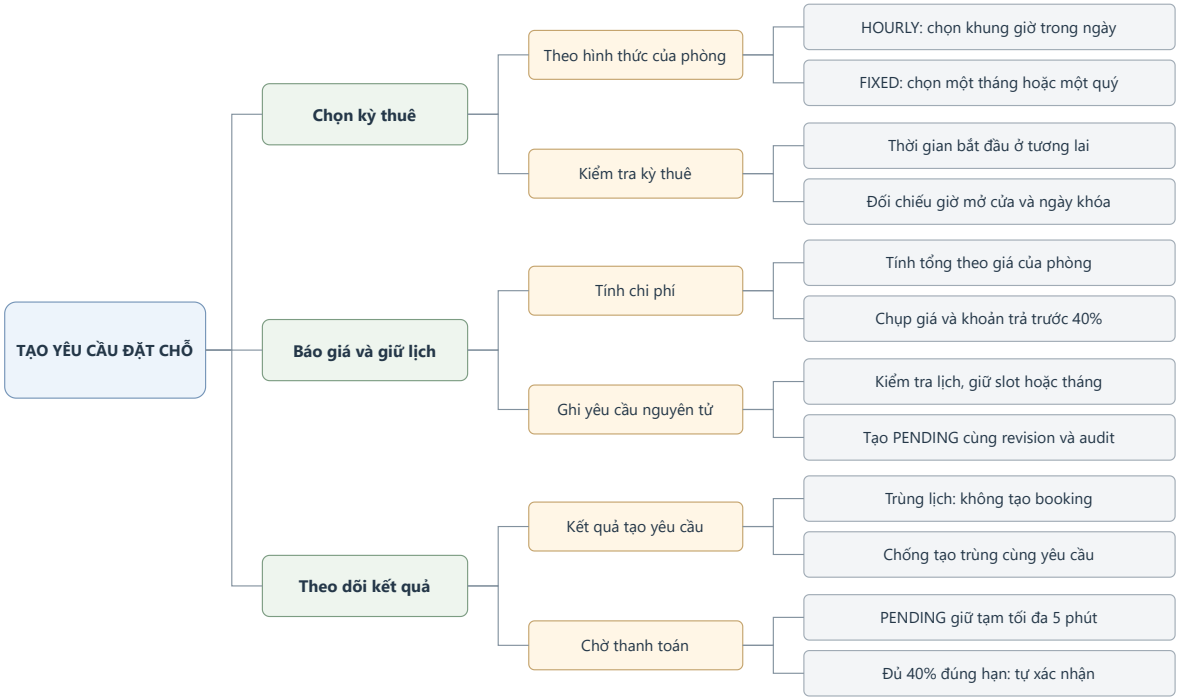
Hình G.1: Phân rã chức năng: tài khoản và phiên.

Khám phá không gian kết hợp tìm kiếm, bộ lọc, bản đồ và thông tin phòng trước khi đặt.



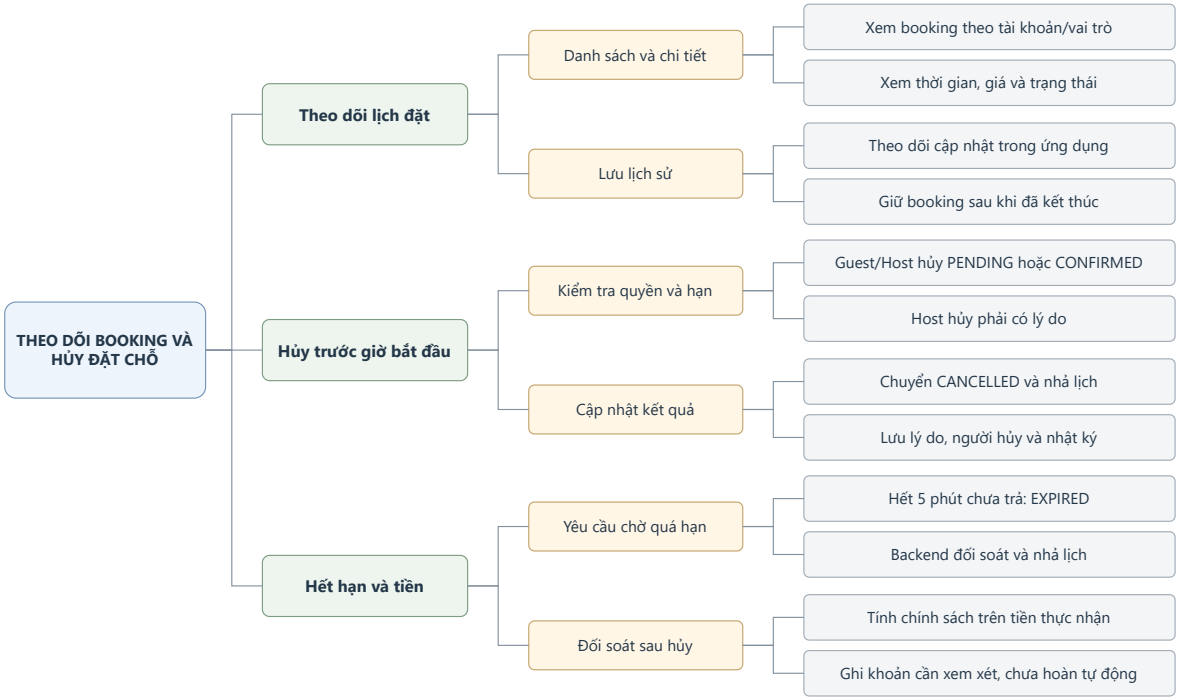
Hình G.2: Phân rã chức năng: khám phá không gian.

Yêu cầu đặt chỗ bao gồm chọn kỳ thuê, kiểm tra khả dụng, chốt giá và mở QR với thời hạn giữ lịch tạm.



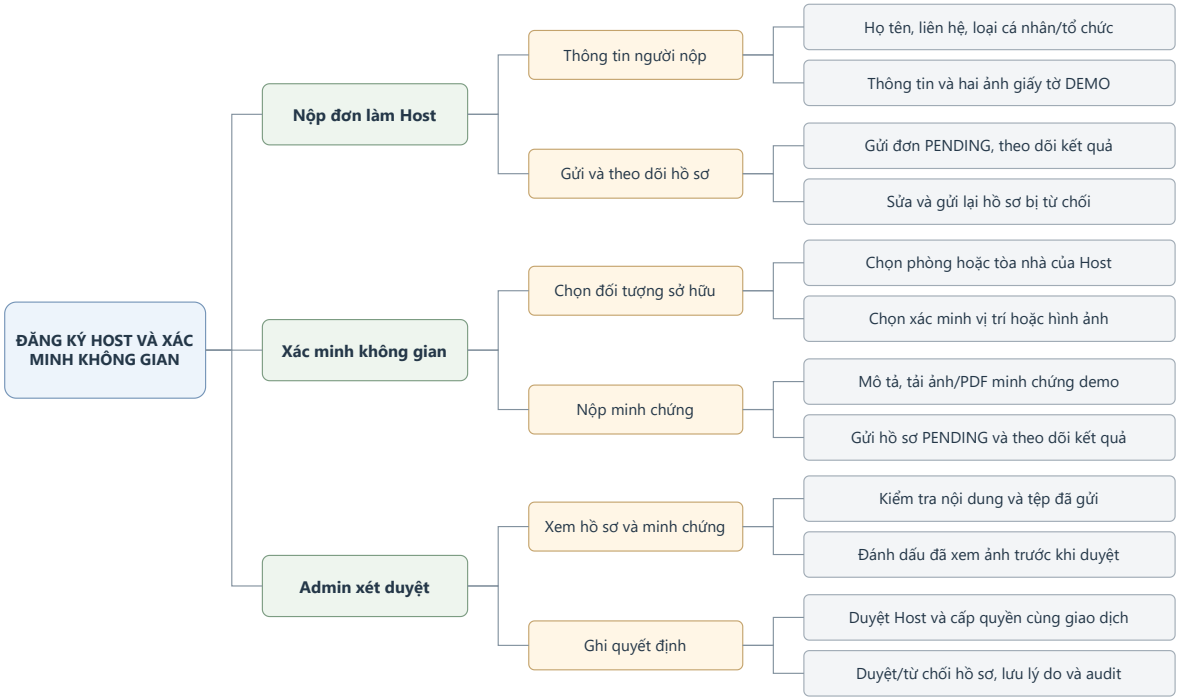
Hình G.3: Phân rã chức năng: tạo yêu cầu đặt chỗ.

Theo dõi lượt thuê phân biệt lịch sử, thời hạn thanh toán và điều kiện hủy; nhả lịch đi cùng cập nhật trạng thái.



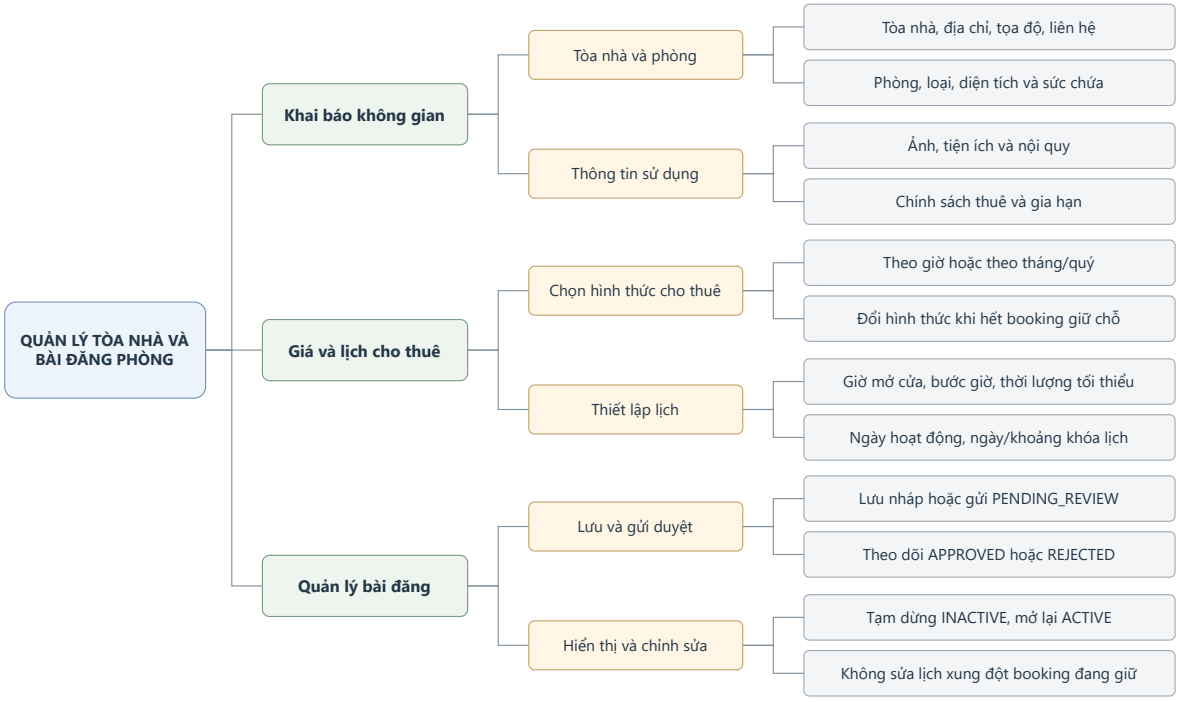
Hình G.4: Phân rã chức năng: lịch đặt và hủy.

Hồ sơ Host được xét duyệt riêng với minh chứng vị trí hoặc hình ảnh của tài sản.



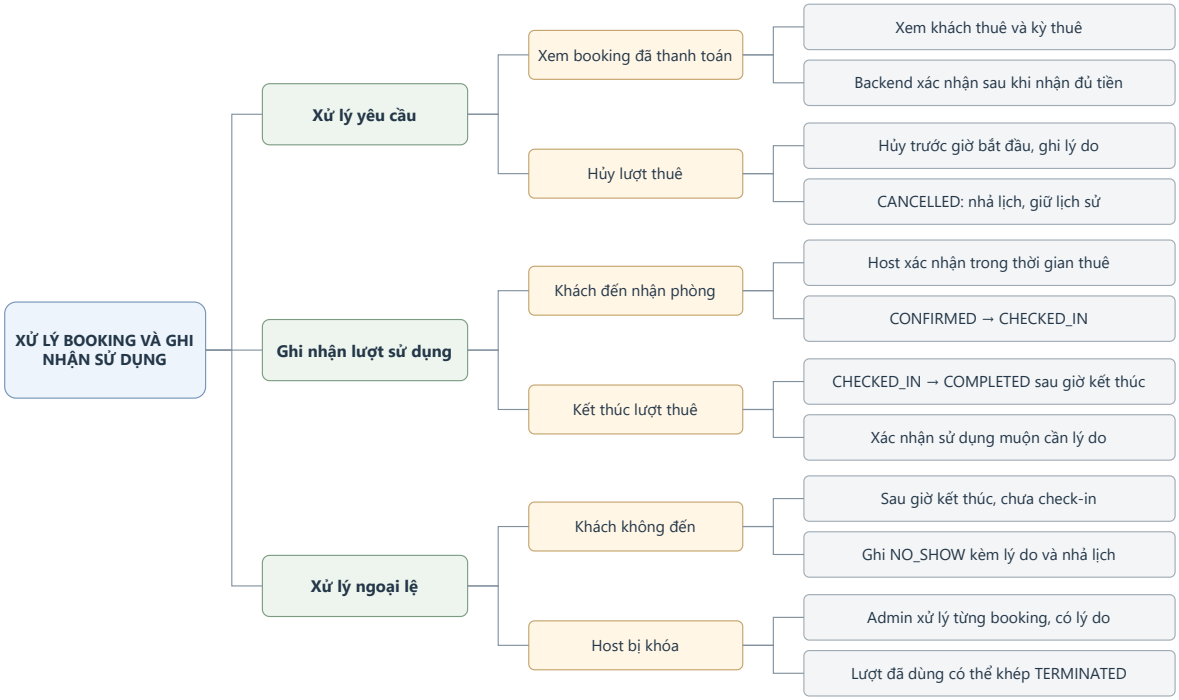
Hình G.5: Phân rã chức năng: hồ sơ host và xác minh.

Quản lý tài sản tách tòa nhà, phòng, cấu hình giá và lịch; công bố phòng phụ thuộc trạng thái xét duyệt và hoạt động.



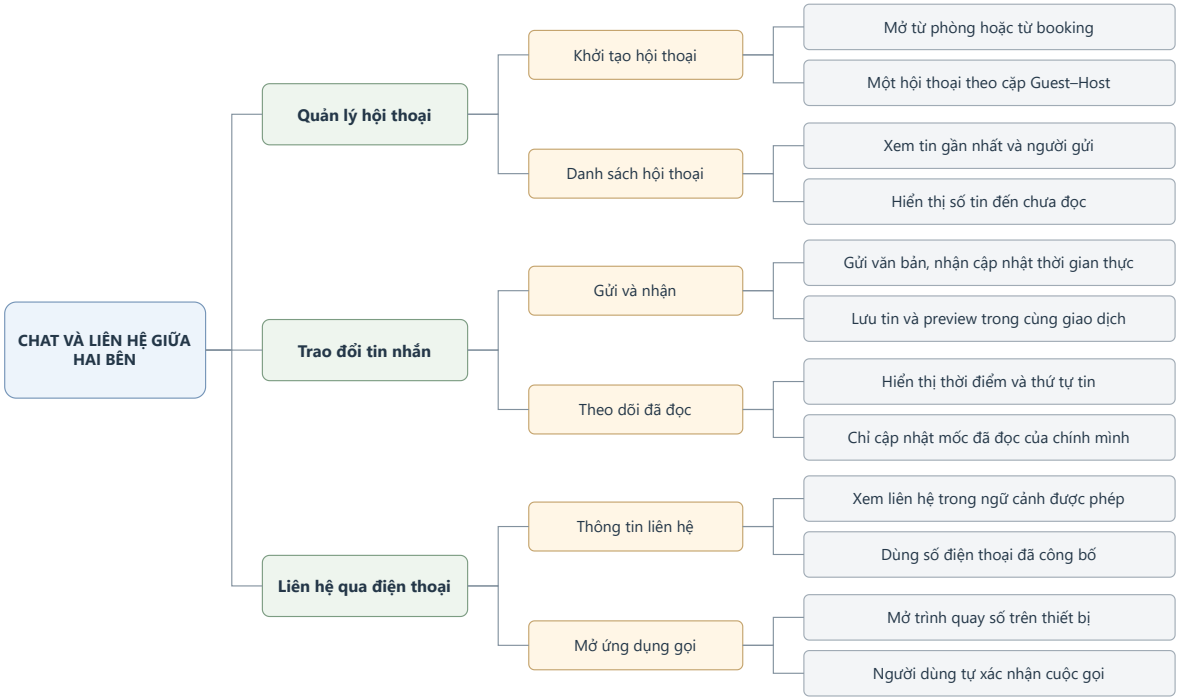
Hình G.6: Phân rã chức năng: quản lý không gian.

Vận hành lượt thuê phân biệt nhận phòng, hoàn tất và không đến, không suy ra đã sử dụng chỉ từ thời gian.



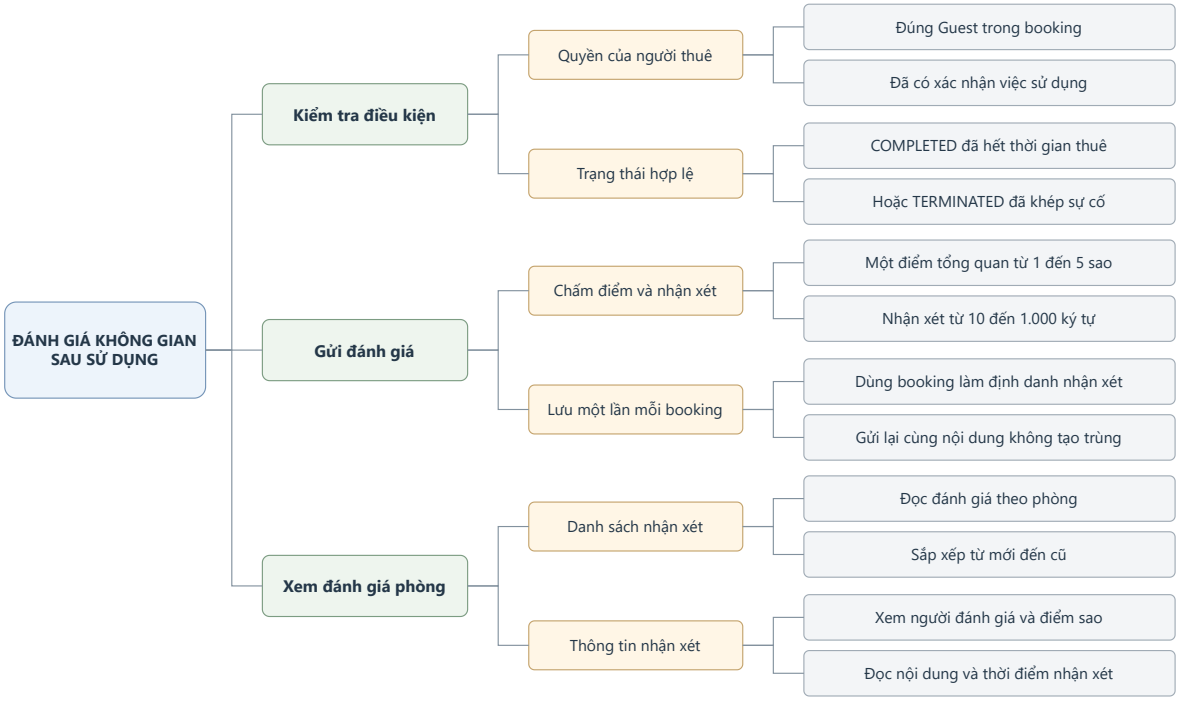
Hình G.7: Phân rã chức năng: theo dõi và sử dụng.

Chat gắn với cặp thành viên, cập nhật tin cuối và mốc đọc; liên hệ không cấp quyền đọc hội thoại cho bên ngoài.



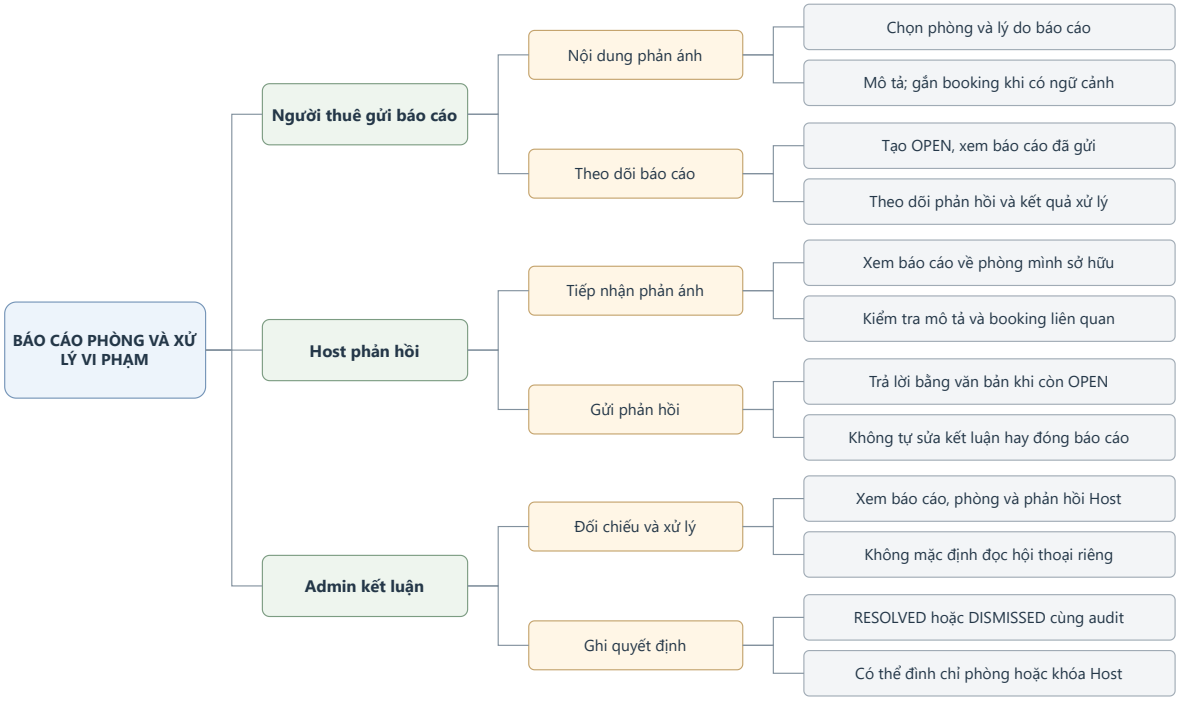
Hình G.8: Phân rã chức năng: chat và liên hệ.

Đánh giá chỉ gắn với lượt đã khép và có xác nhận sử dụng, nhằm tách trải nghiệm thật khỏi lượt không đến.



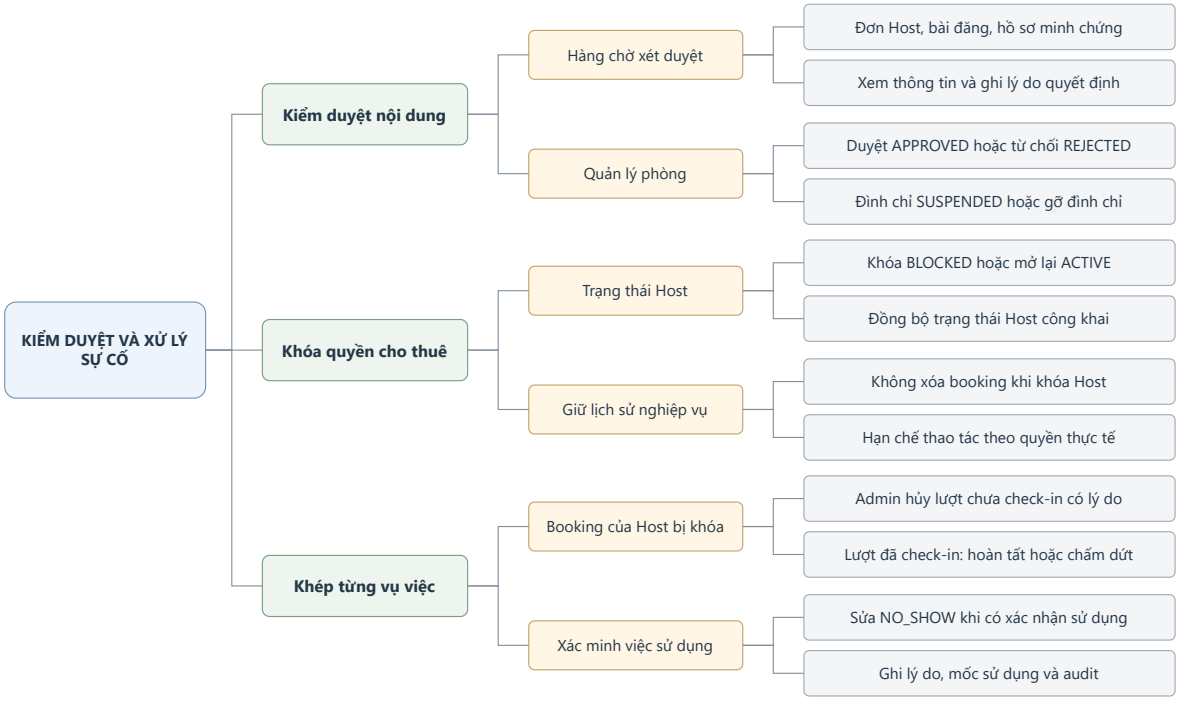
Hình G.9: Phân rã chức năng: đánh giá sau sử dụng.

Báo cáo là hồ sơ xử lý vi phạm có phản hồi của Host và kết luận của Admin, tách với nội dung đánh giá.



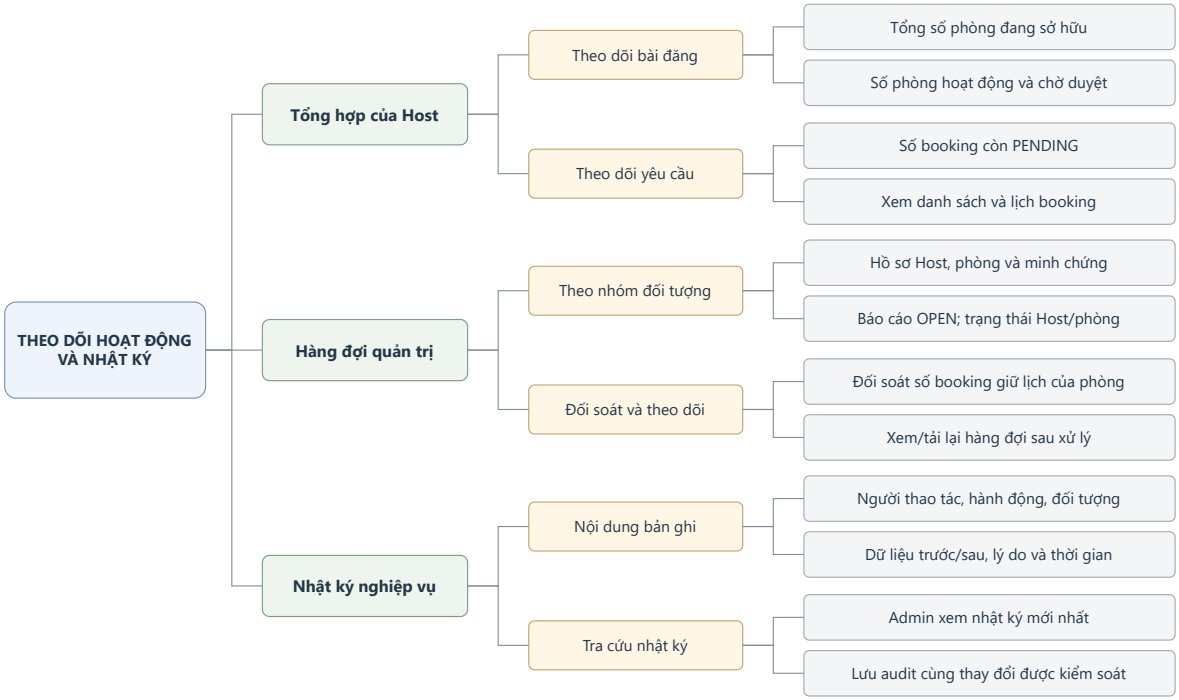
Hình G.10: Phân rã chức năng: báo cáo và phản hồi.

Quản trị bao gồm kiểm duyệt, hạn chế hoạt động và xử lý từng ngoại lệ của booking, có lưu lý do.



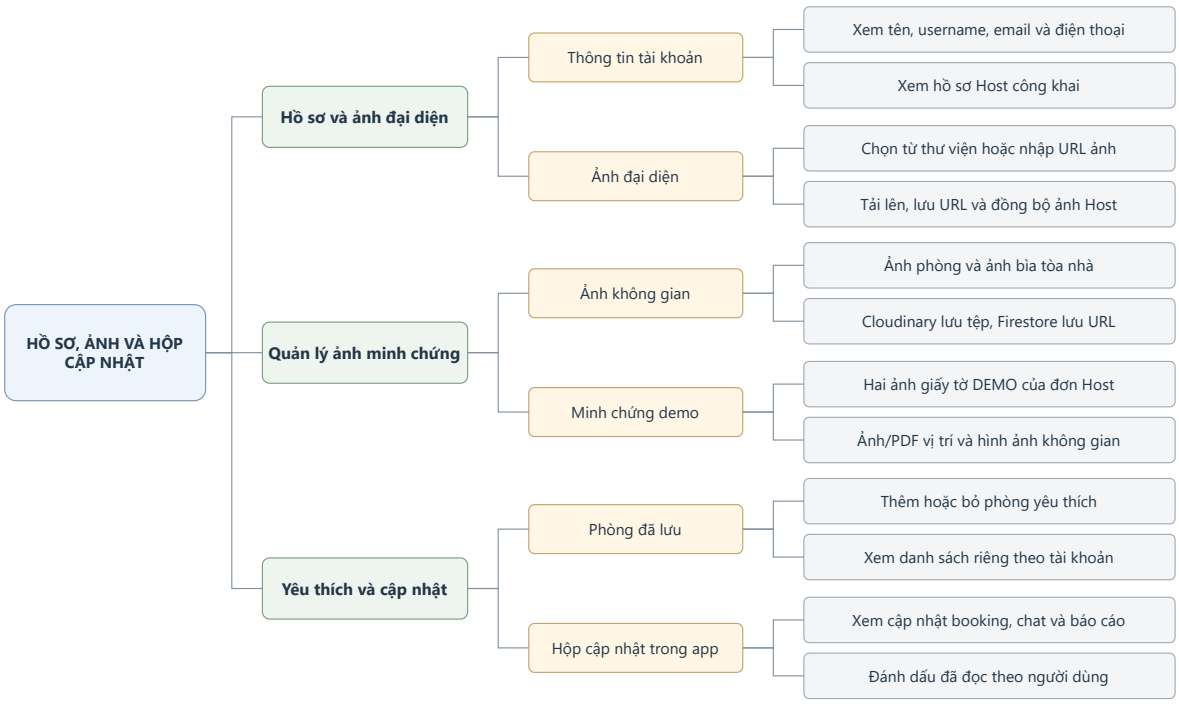
Hình G.11: Phân rã chức năng: quản trị và sự cố.

Theo dõi và nhật ký phục vụ đối chiếu thay đổi; bộ đếm và lịch giữ chỗ phải khớp với các booking đang hiệu lực.



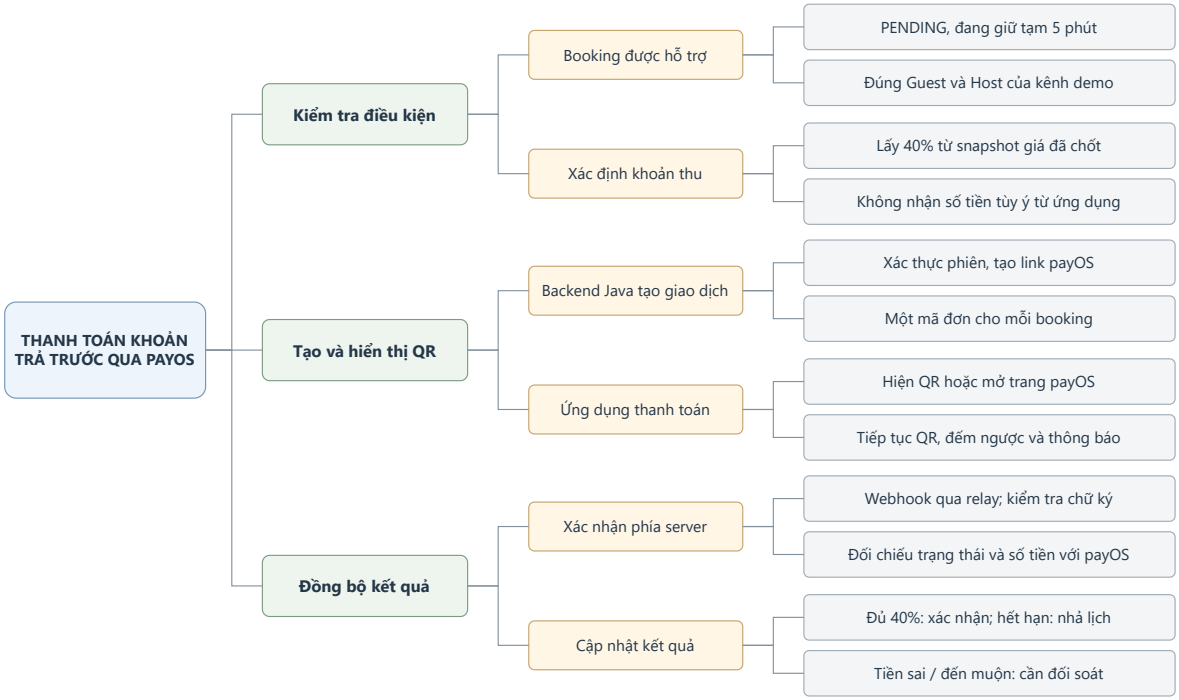
Hình G.12: Phân rã chức năng: theo dõi và nhật ký.

Hồ sơ và thông báo tổng hợp thông tin cá nhân, ảnh đại diện và các cập nhật liên quan đến tài khoản.



Hình G.13: Phân rã chức năng: hồ sơ và thông báo.

Thanh toán demo dùng một kênh payOS: tạo QR, đối soát nhận tiền và cập nhật kết quả; không tự hoàn tiền.



Hình G.14: Phân rã chức năng: thanh toán demo.